# Lógica Computacional — Resumo Teórico

Gonçalo Sousa

Atualizado: Semana 3 (Aulas 1–6)

# Conteúdo

**Aula 1 — Introdução à Lógica e Sintaxe da Lógica Proposicional** **3**
O que é a lógica e porque interessa à informática . . . 3
Proposições . . . 3
Conectivos lógicos . . . 4
Linguagem da lógica proposicional . . . 4
Semântica: valorações e tabelas de verdade . . . 9
Satisfazibilidade, tautologias e contradições . . . 10

**Aula 2 — Consequência Semântica, Equivalência e Conectivos Completos** **11**
A relação de satisfação $\models_v$ . . . 11
Satisfazibilidade de um conjunto de fórmulas . . . 12
Consequência semântica e equivalência . . . 12
Propriedades da relação $\models$ . . . 15
Quantas proposições podem ser simultaneamente verdadeiras . . . 19
Conjuntos completos de conectivos . . . 19
Aplicação: modelar problemas em lógica proposicional . . . 20

**Aula 3 — Formas Normais e Satisfazibilidade** **24**
Literais e forma normal negativa (FNN) . . . 24
Forma normal disjuntiva (FND) . . . 25
Satisfazibilidade de uma FND . . . 27
Forma normal conjuntiva (FNC) . . . 28
Fórmulas de Horn . . . 31

**Aula 4 — Satisfazibilidade: cláusulas e o algoritmo de Davis-Putnam** **35**
O problema SAT . . . 35
Cláusulas . . . 35
Propagação unitária . . . 36
O algoritmo DLL . . . 37

**Aulas 5–6 — Dedução Natural** **40**
Semântica versus dedução . . . 40
Raciocínios válidos e contraexemplos . . . 41
Sistemas dedutivos axiomáticos . . . 41
Dedução natural (DN) e a notação de Fitch . . . 42
Regras da conjunção . . . 43
Regras da disjunção . . . 44
Regras da negação e do absurdo (**F**) . . . 47
Regras da implicação . . . 49
Regras derivadas e estratégias . . . 52
Propriedades de $\vdash$ . . . 55

# Aula 1 — Introdução à Lógica e Sintaxe da Lógica Proposicional

## O que é a lógica e porque interessa à informática

A **lógica** estuda os métodos e princípios que distinguem um raciocínio correto de um incorreto. A lógica formal nasce com Aristóteles (silogismos: regras de inferência como o *modus ponens* — de $A \to B$ e $A$ infere-se $B$), mas só se torna **lógica matemática** a partir do séc. XIX (Boole, Frege, Hilbert, Gödel…), associada a problemas de fundamentação da matemática: teoria de modelos, teoria da dedução (prova) e sistemas axiomáticos.

**Atenção**

Cuidado com silogismos "parecidos" que não são válidos. "Todos os homens são mortais; Sócrates é homem; logo Sócrates é mortal" é válido. Mas "Todos os homens são mortais; Sócrates era mortal; logo todos os homens são Sócrates" **não é válido**, apesar de ter a mesma aparência superficial — a forma do argumento é que importa, não o facto de as frases "soarem" parecidas.

Uma **lógica** (no sentido abstrato) é composta por três partes:

1. uma **linguagem/sintaxe** — um alfabeto e regras para construir *fórmulas*;
2. uma **semântica** — uma noção do que é verdadeiro (pode haver várias interpretações semânticas para a mesma sintaxe);
3. um **sistema de dedução** — um conjunto finito de regras que permite obter fórmulas novas a partir de outras (raciocinar). Interessa que seja **correto** (*sound*): tudo o que se deduz é verdadeiro; e idealmente **completo**: tudo o que é verdadeiro é dedutível (nem sempre é possível ter as duas coisas — é isso que o curso vai explorar com os teoremas de correção e completude; ver "Semântica versus dedução" nas Aulas 5–6).

Neste curso trabalha-se sobretudo com a **lógica clássica** (proposicional e de primeira ordem). Há outras famílias de lógica — lógica de ordem superior, lógicas sub-estruturais (ex: intuicionista, onde $\neg\neg A \not\Leftrightarrow A$, ao contrário do que veremos aqui) e lógicas modais (onde a verdade é parametrizada, ex: por tempo — "Hoje é segunda-feira" só é verdade nalguns "mundos"). É útil ter estas categorias em mente para perceber porque é que a lógica clássica que vamos estudar faz as suposições que faz (por exemplo, todas as proposições são verdadeiras ou falsas — nunca as duas nem nenhuma — o chamado *princípio do terceiro excluído*, que não vale em lógica intuicionista).

A lógica é relevante para a informática em várias frentes: circuitos digitais (lógica booleana), inteligência artificial, autómatos finitos (correspondem a fórmulas de lógica monádica de segunda ordem), XML/documentos estruturados, classes de complexidade caracterizadas por classes de fórmulas, linguagens de programação (programação lógica como Prolog; sistemas de tipos = sistemas dedutivos), verificação e especificação formal (hardware, software, protocolos de segurança), dedução automática (SAT/SMT solvers) e provadores de teoremas assistidos (Coq, Isabelle, Agda).

## Proposições

**Definição — Proposição**

Uma **proposição** é uma frase declarativa à qual se pode atribuir um valor de verdade, **V** (verdadeiro) ou **F** (falso) — e apenas um dos dois. Frases que não afirmam nada com um valor de verdade definido (perguntas, ordens, expressões sem verbo, ou frases com variáveis livres cujo valor de verdade depende de algo não especificado) **não são proposições**.

**Exemplo resolvido — Exercício 1.1 (lab, proplogic.pdf) — 5 das 8 alíneas, escolhidas para não repetir a mesma ideia**

Para cada frase, indicar se é proposição e, sendo, o seu valor de verdade (as 8 alíneas do enunciado testam só 5 ideias distintas — ver nota a seguir à caixa):
**(a)** "O Porto é a capital de Portugal." — É proposição. **F** (a capital é Lisboa). *(caso base: frase factual simples, com valor de verdade definido.)*
**(d)** "$1+2+3$" — **Não é proposição**: é uma expressão/termo (um número), não uma afirmação. Não diz nada que possa ser verdadeiro ou falso.
**(c)** "Hei de comparar-te a um dia de verão?" — **Não é proposição**: é uma pergunta (interrogativa), não uma afirmação — não tem valor de verdade (nota que isto é um motivo *diferente* do de (d): ali era por não ser uma afirmação de todo, aqui é por ser uma pergunta).
**(g)** "$x+y$ é um número par." — **Não é proposição**: $x$ e $y$ são *variáveis livres* — o valor de verdade depende de que valores lhes atribuirmos (é verdade para $x=1, y=1$, falso para $x=1, y=2$). A frase só passa a ter um valor de verdade fixo depois de fixarmos $x$ e $y$.
**(h)** "Existe um $x$ tal que $7+x$ é um número par." — É proposição, e **V**: apesar de ter uma variável, o quantificador "existe um $x$" **fecha** a variável — a frase inteira já não depende de nenhum $x$ externo (basta $x=1$: $7+1=8$ é par). Compara com (g): a diferença entre uma variável livre (não é proposição) e uma variável quantificada (é proposição) só fica completamente formal em lógica de primeira ordem, mais à frente no curso — mas já vale a pena notar a distinção agora.

**Atenção**

As alíneas (b), (e) e (f) do Exercício 1.1 ficam de fora de propósito — testam exatamente a mesma ideia de (a) (frase factual simples, decidir V/F), só com outro assunto. Não precisas de as fazer; se quiseres confirmar que percebeste, é literalmente o mesmo passo de (a) com números diferentes.

## Conectivos lógicos

Cada proposição simples é representada por uma **variável proposicional** $(p, q, r, ...)$, e proposições compostas obtêm-se combinando-as com **conectivos**:

| Conectivo | Símbolo | Aridade | Outros símbolos equivalentes |
|---|---|---|---|
| Conjunção ("e") | $\wedge$ | 2 | `&`, `&&`, $\cdot$ |
| Disjunção ("ou") | $\vee$ | 2 | `\|`, `+` |
| Negação ("não") | $\neg$ | 1 | $\sim$, `~`, `!` |
| Implicação ("se…então") | $\rightarrow$ | 2 | $\Rightarrow$, $\supset$ |

## Linguagem da lógica proposicional

**Definição — Alfabeto**

A linguagem da lógica proposicional é construída a partir dos símbolos primitivos:

- um conjunto **numerável** de variáveis proposicionais $\mathcal{V}_{Prop} = \{p, q, r, \dots, p_1, \dots\}$;
- os conectivos lógicos $\wedge, \vee, \neg, \rightarrow$;
- os parêntesis `(` e `)`.

**Definição — Fórmula bem formada**

O conjunto das **fórmulas** $(\phi, \psi, \theta, ...)$ é definido indutivamente:

1. uma variável proposicional $p$ é uma fórmula;
2. se $\phi$ é fórmula, então $(\neg\phi)$ é fórmula;
3. se $\phi$ e $\psi$ são fórmulas, então $(\phi \wedge \psi)$, $(\phi \vee \psi)$ e $(\phi \to \psi)$ são fórmulas.

Nada mais é fórmula. Exemplo: $((p \wedge (\neg p)) \to (\neg(p \wedge (q \vee r))))$.

Esta definição indutiva é o que permite desenhar a **árvore sintática** de uma fórmula: a raiz é o conectivo principal, e os seus filhos são as fórmulas que ele combina (recursivamente, até chegar às variáveis, que são as folhas).

### Convenção para omitir parêntesis

Escrever todos os parêntesis torna as fórmulas ilegíveis, por isso usa-se uma convenção de precedência (tal como em aritmética, onde $\times$ tem prioridade sobre $+$):

- os parêntesis mais exteriores podem ser omitidos;
- $\neg$ tem prioridade sobre $\wedge$;
- $\wedge$ tem prioridade sobre $\vee$;
- $\vee$ tem prioridade sobre $\to$;
- $\wedge$ e $\vee$ são **associativos à esquerda**;
- $\to$ é **associativo à direita**.

**Exemplo resolvido**

- $\phi \wedge \psi \vee \theta$ abrevia $((\phi \wedge \psi) \vee \theta)$ (o $\wedge$ "agarra" primeiro).
- $p \wedge \neg p \to \neg(p \wedge (q \vee r))$ corresponde a $((p \wedge (\neg p)) \to (\neg(p \wedge (q \vee r))))$.
- $p \to q \to r$ corresponde a $p \to (q \to r)$ (associatividade à direita do $\to$: lê-se "de dentro para fora", começando pelo lado direito).

**Definição — Subfórmula**

Uma **subfórmula imediata** de $\psi$ é: (1) nenhuma, se $\psi$ é uma variável; (2) $\phi$, se $\psi = \neg\phi$; (3) $\phi$ e $\psi'$, se $\psi = \phi \wedge \psi'$, $\phi \vee \psi'$ ou $\phi \to \psi'$.
$\phi$ é **subfórmula** de $\psi$ se $\phi$ é subfórmula imediata de $\psi$, ou se existe $\theta$ tal que $\phi$ é subfórmula de $\theta$ e $\theta$ é subfórmula de $\psi$ (fecho transitivo das subfórmulas imediatas). Por convenção, $\psi$ é sempre subfórmula de si própria.

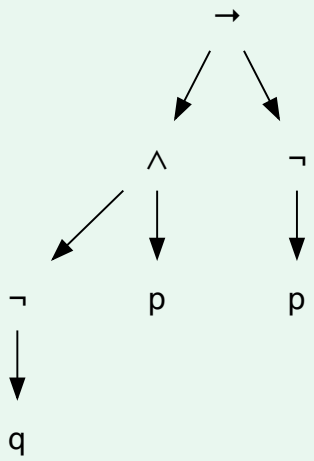
O exercício seguinte junta a convenção de precedência, a árvore sintática e a noção de subfórmula — a melhor forma de treinar os três em conjunto é desenhar a árvore completa e depois ler as subfórmulas diretamente dos nós dessa árvore (cada nó interno, incluindo a raiz, é uma subfórmula; as folhas são as variáveis).

**Exemplo resolvido — Exercício 1.3(a) (lab, proplogic.pdf)**

**Fórmula:** $p \wedge \neg q \rightarrow \neg p$.
**Passo 1 — reintroduzir os parêntesis pela convenção de precedência.** $\neg$ agarra primeiro: $\neg q$ e $\neg p$. Depois $\wedge$: $p \wedge (\neg q)$. Por fim $\rightarrow$ (o conectivo de menor prioridade, por isso é a raiz):

$$(p \wedge (\neg q)) \rightarrow (\neg p)$$

**Passo 2 — árvore sintática** (raiz = conectivo principal, $\rightarrow$):



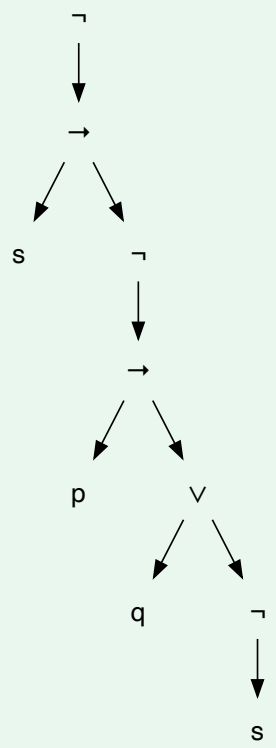

**Passo 3 — lista de subfórmulas** (lidas diretamente dos nós da árvore, da raiz para as folhas): $p \wedge \neg q \rightarrow \neg p$ (a própria fórmula), $p \wedge \neg q$, $\neg p$, $p$, $\neg q$, $q$.

**Exemplo resolvido — Exercício 1.3(c) (lab, proplogic.pdf)**

**Fórmula:** $\neg(s \to (\neg(p \to (q \vee \neg s))))$.
Aqui os parêntesis já vêm quase todos explícitos no enunciado — só falta decompor "de fora para dentro" para identificar cada conectivo principal. Vamos dar um nome a cada subfórmula à medida que aparece, de dentro para fora (é a ordem inversa da leitura, mas é a ordem em que a árvore se constrói):

- $A = q \vee \neg s$ (disjunção de $q$ com $\neg s$)
- $B = p \to A$
- $C = \neg B$
- $D = s \to C$
- $E = \neg D$ (a fórmula toda)

**Árvore sintática:**



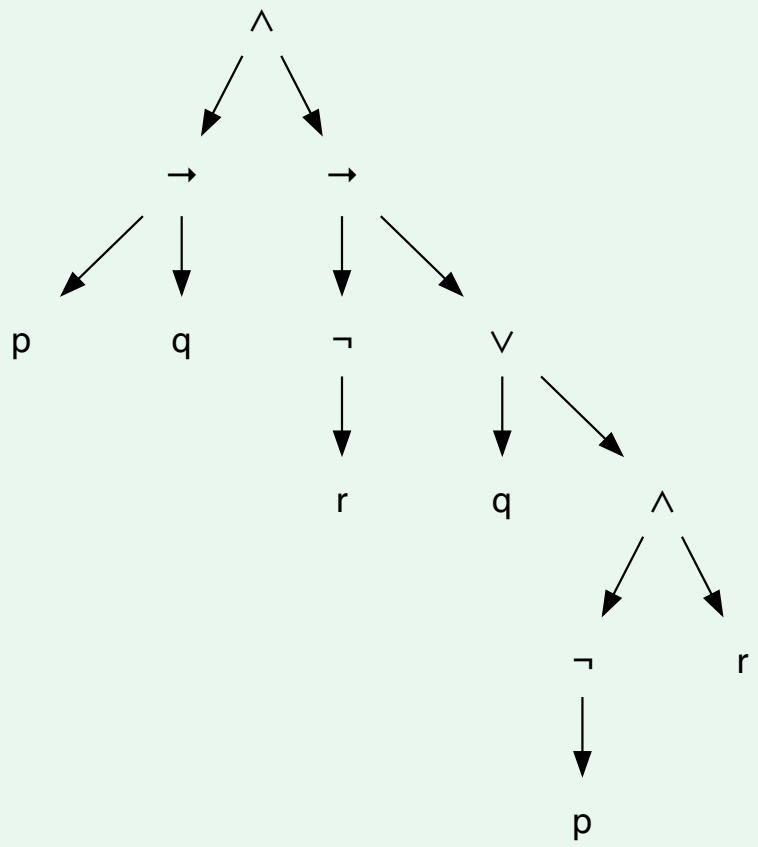

**Subfórmulas:** $E$ (a própria fórmula), $D = s \to \neg(p \to (q \vee \neg s))$, $s$, $C = \neg(p \to (q \vee \neg s))$, $B = p \to (q \vee \neg s)$, $p$, $A = q \vee \neg s$, $q$, $\neg s$, $s$ (repara que $s$ aparece **duas vezes** na árvore, como dois nós diferentes — mas é a mesma fórmula, só se lista uma vez no conjunto de subfórmulas).

**Exemplo resolvido — Exercício 1.3(e) (lab, proplogic.pdf)**

**Fórmula:** $(p \to q) \wedge (\neg r \to (q \vee (\neg p \wedge r)))$.
De novo os parêntesis já resolvem a ambiguidade; só falta identificar a estrutura. O conectivo principal é o $\wedge$ mais exterior, que combina dois ramos:

- ramo esquerdo: $L = p \to q$
- ramo direito: $M = \neg r \to M_2$, onde $M_2 = q \vee M_1$ e $M_1 = \neg p \wedge r$

**Árvore sintática:**




**Subfórmulas:** a fórmula toda, $L = p \to q$, $p$, $q$, $M = \neg r \to (q \vee (\neg p \wedge r))$, $\neg r$, $r$, $M_2 = q \vee (\neg p \wedge r)$, $q$, $M_1 = \neg p \wedge r$, $\neg p$, $p$, $r$.

**Atenção**

Nota adicional (não estava explícito nos slides, mas é essencial para não errar nos exercícios): quando reconstróis os parêntesis pela convenção de precedência, faz sempre isso **antes** de tentar desenhar a árvore — tentar fazer as duas coisas ao mesmo tempo é a fonte mais comum de erros. Também repara que o conectivo **principal** de uma fórmula (a raiz da árvore) é sempre o de **menor prioridade** entre os que aparecem "soltos" (não dentro de parêntesis explícitos) — por isso em 1.3(a) a raiz é $\to$ e não $\wedge$, mesmo aparecendo $\wedge$ mais à esquerda na leitura da fórmula.

**Pratica agora — Linguagem e árvores sintáticas (proplogic.pdf)**

- **1.2** — traduzir cada fórmula para português. É mecânico: lê cada conectivo com a tabela da secção "Conectivos lógicos" ($\rightarrow$ = "se… então"). Atenção à (d) vs. (e) vs. (f): são três implicações diferentes, não digas o mesmo por palavras diferentes.
- **1.3 (b)** e **(d)** — árvore sintática e subfórmulas, mesma técnica de (a)/(c)/(e) acima. Na (b) compara com a (a): os parêntesis mudam o conectivo principal.

*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

## Semântica: valorações e tabelas de verdade

**Definição — Valoração (atribuição de valores de verdade)**

Uma **valoração** é uma função $v : \mathcal{V}_{Prop} \rightarrow \{V, F\}$ que atribui um valor de verdade a cada variável proposicional. Estende-se ao conjunto das fórmulas por indução na estrutura:

1. $v(p)$ já está definido, para $p \in \mathcal{V}_{Prop}$;
2. $v(\neg\phi) = V$ se $v(\phi) = F$, e $v(\neg\phi) = F$ se $v(\phi) = V$;
3. $v(\phi \wedge \psi) = V$ sse $v(\phi) = V$ e $v(\psi) = V$ (senão $F$);
4. $v(\phi \vee \psi) = V$ sse $v(\phi) = V$ ou $v(\psi) = V$ (senão $F$);
5. $v(\phi \rightarrow \psi) = F$ sse $v(\phi) = V$ e $v(\psi) = F$ (senão $V$).

**Atenção**

O caso mais contra-intuitivo é a implicação: $\phi \rightarrow \psi$ só é **falsa** quando o antecedente é verdadeiro e o consequente é falso. Em particular, sempre que o antecedente $\phi$ é **falso**, a implicação inteira é **verdadeira**, independentemente do consequente ("de uma premissa falsa pode concluir-se o que quiser" — *ex falso quodlibet*). Isto vai ser usado constantemente nas provas "sem construir a tabela de verdade" (Exercício 1.14, Aula 2).

| $\phi$ | $\neg\phi$ |
|---|---|
| F | V |
| V | F |

| $\phi$ | $\psi$ | $\phi \wedge \psi$ |
|---|---|---|
| F | F | F |
| F | V | F |
| V | F | F |
| V | V | V |

| $\phi$ | $\psi$ | $\phi \vee \psi$ |
|---|---|---|
| F | F | F |
| F | V | V |
| V | F | V |
| V | V | V |

| $\phi$ | $\psi$ | $\phi \rightarrow \psi$ |
|---|---|---|
| F | F | V |
| F | V | V |
| V | F | F |
| V | V | V |

Uma fórmula $\phi$ com $n$ variáveis proposicionais tem $2^n$ valorações possíveis (cada variável pode ser $V$ ou $F$, independentemente das outras) — constrói-se uma **tabela de verdade** com uma linha por valoração, calculando o valor de $\phi$ coluna a coluna (dos operadores mais "internos" para o mais "externo", exatamente como na árvore sintática).

**Exemplo resolvido — Exercício 1.4(b) (lab, proplogic.pdf)**

Construir a tabela de verdade de $\neg q \wedge (p \to q)$ (2 variáveis, logo 4 linhas):

| $p$ | $q$ | $\neg q$ | $p \to q$ | $\neg q \wedge (p \to q)$ |
|---|---|---|---|---|
| V | V | F | V | F |
| V | F | V | F | F |
| F | V | F | V | F |
| F | F | V | V | **V** |

A fórmula só é verdadeira quando $p$ e $q$ são ambos falsos. Isto não é coincidência: por equivalências semânticas (ver mais abaixo), $\neg q \wedge (p \to q) \Leftrightarrow \neg q \wedge (\neg p \vee q) \Leftrightarrow (\neg q \wedge \neg p) \vee (\neg q \wedge q) \Leftrightarrow (\neg p \wedge \neg q) \vee F \Leftrightarrow \neg p \wedge \neg q$ — exatamente o que a tabela mostra.

**Exemplo resolvido — Exercício 1.4(c) (lab, proplogic.pdf)**

Construir a tabela de verdade de $(p \wedge q) \to \neg(p \vee q)$:

| $p$ | $q$ | $p \wedge q$ | $p \vee q$ | $\neg(p \vee q)$ | $(p \wedge q) \to \neg(p \vee q)$ |
|---|---|---|---|---|---|
| V | V | V | V | F | **F** |
| V | F | F | V | F | **V** |
| F | V | F | V | F | **V** |
| F | F | F | F | V | **V** |

Só é falsa na linha $p = V, q = V$: aí o antecedente $p \wedge q$ é verdadeiro mas o consequente $\neg(p \vee q)$ é falso (porque $p \vee q$ é verdadeiro). Em todas as outras linhas o antecedente $p \wedge q$ já é falso, o que torna a implicação automaticamente verdadeira (ver nota acima sobre a implicação).

**Pratica agora — Tabelas de verdade (proplogic.pdf)**

- **1.4 (a)** e **(d)** — tabelas de verdade, mesma técnica de (b)/(c) acima. Na (d), antes de acabar, repara em que linhas o antecedente é verdadeiro: é aí que tudo se decide.

*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

## Satisfazibilidade, tautologias e contradições

**Definição — Satisfazibilidade, tautologia, contradição**

Uma fórmula $\phi$ é:

- **satisfazível** se existe uma valoração $v$ tal que $v(\phi) = V$ — escreve-se $\models_v \phi$ e diz-se que $v$ **satisfaz** $\phi$;
- **tautologia** se **todas** as valorações $v$ dão $v(\phi) = V$ — escreve-se $\models \phi$ (ex: $\models p \vee \neg p$, o princípio do terceiro excluído);
- **contradição** se **todas** as valorações $v$ dão $v(\phi) = F$ — escreve-se $\not\models \phi$ (ex: $\not\models p \wedge \neg p$).

Uma fórmula é **insatisfazível** sse é uma contradição. Uma fórmula satisfazível mas não tautologia diz-se por vezes **contingente**: é verdadeira nalgumas valorações e falsa noutras.

# Aula 2 — Consequência Semântica, Equivalência e Conectivos Completos

## A relação de satisfação $\models_v$

A extensão de $v$ às fórmulas (aula anterior) pode ser reescrita como uma relação $\models_v$, definida indutivamente na estrutura da fórmula — é apenas outra notação para a mesma coisa, mas é a notação usada daqui em diante:

**Definição — Relação de satisfação**

Dada uma valoração $v$, a relação $\models_v$ define-se por:

1. $\models_v p$ sse $v(p) = V$;
2. $\models_v \neg\phi$ sse $\not\models_v \phi$;
3. $\models_v \phi \wedge \psi$ sse $\models_v \phi$ e $\models_v \psi$;
4. $\models_v \phi \vee \psi$ sse $\models_v \phi$ ou $\models_v \psi$;
5. $\models_v \phi \to \psi$ sse $\not\models_v \phi$ ou $\models_v \psi$.

Diz-se que $v$ **satisfaz** $\phi$ se $\models_v \phi$.

**Exemplo resolvido**

Com $v(p) = V, v(q) = v(r) = F$, para saber se $\models_v (p \to (q \vee r)) \vee (r \to \neg p)$ basta que **um** dos dois lados da disjunção seja satisfeito. O lado direito, $r \to \neg p$, é satisfeito porque $\not\models_v r$ (implicação com antecedente falso). Logo $\models_v (p \to (q \vee r)) \vee (r \to \neg p)$ — nem foi preciso avaliar o lado esquerdo.

**Exemplo resolvido — Exercício 1.5(b) (lab, proplogic.pdf)**

Com $v(p) = V$ e $v(q) = F$, verificar se $\models_v (p \vee \neg q) \wedge (\neg p \wedge q)$.

O segundo conjunto, $\neg p \wedge q$: temos $v(p) = V$ logo $\not\models_v \neg p$ (já falha aqui). Como um conjunto ($\wedge$) precisa dos **dois** lados satisfeitos e um deles já falha, $\not\models_v \neg p \wedge q$. Logo, mesmo sem avaliar o primeiro conjunto, a conjunção inteira falha: $\not\models_v (p \vee \neg q) \wedge (\neg p \wedge q)$.

**Exemplo resolvido — Exercício 1.5(e) (lab, proplogic.pdf)**

Com $v(p) = V$ e $v(q) = F$, verificar se $\models_v ((p \to q) \to p) \to p$.

$p \to q$: $v(p) = V, v(q) = F$, logo $\not\models_v p \to q$ (antecedente verdadeiro, consequente falso — o único caso em que a implicação falha). $(p \to q) \to p$: antecedente falso ($\not\models_v p \to q$), logo esta implicação é automaticamente satisfeita: $\models_v (p \to q) \to p$. Fórmula toda: $((p \to q) \to p) \to p$: antecedente satisfeito ($\models_v (p \to q) \to p$) **e** consequente satisfeito ($\models_v p$, porque $v(p) = V$) — logo $\models_v ((p \to q) \to p) \to p$.

**Frequente em exame**

$((p \to q) \to p) \to p$ é a **Lei de Peirce**, e é de facto uma **tautologia** (é verdadeira para *qualquer* $v$, não só para o $v$ deste exercício — podes confirmar construindo a tabela completa com as 4 valorações de $p, q$). É um resultado clássico e frequentemente pedido: é um dos exemplos-padrão de tautologia que **não é válida em lógica intuicionista** (relembra a nota da Aula 1 sobre lógicas sub-estruturais) — distingue-a da lógica clássica que estamos a estudar.

**Atenção**

Nota adicional — armadilha comum: **não confundir** $(p \to q) \to p$ (que aparece na Lei de Peirce acima) com $(q \to p) \to p$ (parecida, mas com $p$ e $q$ trocados no antecedente). Ao contrário da primeira, $(q \to p) \to p$ **não** é tautologia — falha, por exemplo, com $v(p) = F$: nesse caso $q \to p = \neg q$ (verdadeiro só se $q$ for falso), e se $v(q) = F$ então $q \to p$ é verdadeiro, tornando $(q \to p) \to p$ igual a $V \to F = F$. A posição exata de cada variável na fórmula importa — nunca assumas que uma fórmula "parecida" com uma tautologia conhecida também o é.

**Pratica agora — A relação $\models_v$ (proplogic.pdf)**

- **1.5 (a)**, **(c)** e **(d)** — com $v(p) = V$, $v(q) = F$, avalia de dentro para fora, como em (b)/(e) acima.

*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

## Satisfazibilidade de um conjunto de fórmulas

**Definição — Satisfazibilidade de um conjunto**

Seja $\Gamma$ um conjunto de fórmulas. Uma valoração $v$ **satisfaz** $\Gamma$ (notação $\models_v \Gamma$) se satisfaz **todas** as fórmulas de $\Gamma$: $\forall \psi \in \Gamma,\ \models_v \psi$. $\Gamma$ é **satisfazível** se existe alguma valoração que o satisfaz.

## Consequência semântica e equivalência

**Definição — Consequência semântica ($\models$)**

$\phi$ é **consequência semântica** de $\Gamma$ (notação $\Gamma \models \phi$) se **toda** valoração $v$ que satisfaz $\Gamma$ também satisfaz $\phi$.

Casos especiais de notação:

- $\emptyset \models \phi$ escreve-se $\models \phi$ — equivale a $\phi$ ser tautologia (nenhuma condição sobre $v$: tem de valer sempre).
- Se $\Gamma = \{\psi\}$, escreve-se $\psi \models \phi$.
- Se $\psi \models \phi$ **e** $\phi \models \psi$, $\psi$ e $\phi$ dizem-se **semanticamente equivalentes**, escreve-se $\psi \Leftrightarrow \phi$ (têm exatamente o mesmo valor de verdade em qualquer valoração).

**Definição — Leis de equivalência semântica**

| Lei | Equivalência |
|---|---|
| Comutatividade | $\phi \wedge \psi \Leftrightarrow \psi \wedge \phi$ ; $\phi \vee \psi \Leftrightarrow \psi \vee \phi$ |
| Lei de De Morgan | $\neg(\phi \wedge \psi) \Leftrightarrow \neg\phi \vee \neg\psi$ ; $\neg(\phi \vee \psi) \Leftrightarrow \neg\phi \wedge \neg\psi$ |
| Associatividade | $(\phi \wedge \psi) \wedge \theta \Leftrightarrow \phi \wedge (\psi \wedge \theta)$ ; idem para $\vee$ |
| Idempotência | $\phi \vee \phi \Leftrightarrow \phi$ ; $\phi \wedge \phi \Leftrightarrow \phi$ |
| Distributividade | $(\phi \wedge \psi) \vee \theta \Leftrightarrow (\phi \vee \theta) \wedge (\psi \vee \theta)$ ; $(\phi \vee \psi) \wedge \theta \Leftrightarrow (\phi \wedge \theta) \vee (\psi \wedge \theta)$ |
| Dupla negação | $\neg\neg\phi \Leftrightarrow \phi$ |
| Implicação | $\phi \to \psi \Leftrightarrow \neg\phi \vee \psi$ |

Estas leis funcionam como uma "álgebra booleana": permitem transformar uma fórmula noutra equivalente sem ter de recorrer à tabela de verdade — substituindo subfórmulas pelo seu equivalente, passo a passo (tal como se faz álgebra normal, mas com $\wedge, \vee, \neg$ em vez de $\times, +$).

**Atenção — Leis extra para simplificar**

Nota adicional (não estão na tabela dos slides nem na do enunciado, mas são precisas para **simplificar** uma fórmula até se ver o que ela é — cada uma confirma-se com uma tabela de 2 linhas). $V$ e $F$ representam uma fórmula sempre verdadeira e uma sempre falsa:

| Lei | Equivalência |
|---|---|
| Complementaridade | $\phi \vee \neg\phi \Leftrightarrow V$ ; $\phi \wedge \neg\phi \Leftrightarrow F$ |
| Elemento neutro | $\phi \vee F \Leftrightarrow \phi$ ; $\phi \wedge V \Leftrightarrow \phi$ |
| Elemento absorvente | $\phi \vee V \Leftrightarrow V$ ; $\phi \wedge F \Leftrightarrow F$ |
| Absorção | $\phi \vee (\phi \wedge \psi) \Leftrightarrow \phi$ ; $\phi \wedge (\phi \vee \psi) \Leftrightarrow \phi$ |

**Exemplo resolvido — Exercício 1.6(a) (lab, proplogic.pdf)**

Mostrar que $\neg(p \to q) \Leftrightarrow p \wedge \neg q$, usando as leis (sem tabela de verdade):

$$\neg(p \to q) \overset{\text{(implicação)}}{\Leftrightarrow} \neg(\neg p \vee q) \overset{\text{(De Morgan)}}{\Leftrightarrow} \neg\neg p \wedge \neg q \overset{\text{(dupla negação)}}{\Leftrightarrow} p \wedge \neg q$$

Cada passo substitui uma subfórmula pelo seu equivalente segundo uma única lei da tabela — é assim que se lê/escreve qualquer prova por equivalências.

**Exemplo resolvido — Exercício 1.6(e) (lab, proplogic.pdf)**

Mostrar que $(p \vee q) \to r \Leftrightarrow (p \to r) \wedge (q \to r)$:

$$(p \vee q) \to r \overset{\text{(implicação)}}{\Leftrightarrow} \neg(p \vee q) \vee r \overset{\text{(De Morgan)}}{\Leftrightarrow} (\neg p \wedge \neg q) \vee r$$

Agora aplica-se a distributividade — $(\phi \wedge \psi) \vee \theta \Leftrightarrow (\phi \vee \theta) \wedge (\psi \vee \theta)$ — com $\phi = \neg p$, $\psi = \neg q$, $\theta = r$:

$$(\neg p \wedge \neg q) \vee r \overset{\text{(distributividade)}}{\Leftrightarrow} (\neg p \vee r) \wedge (\neg q \vee r) \overset{\text{(implicação, 2x)}}{\Leftrightarrow} (p \to r) \wedge (q \to r)$$

**Atenção**

Guarda a equivalência do Exercício 1.6(e) — $(p \vee q) \to r \Leftrightarrow (p \to r) \wedge (q \to r)$ — porque serve para **confirmar** o Exercício 1.13(d) mais abaixo: como $(p \to r) \wedge (q \to r)$ implica $(p \to r) \vee (q \to r)$, a consequência tem de valer. Em exame, porém, a 1.13(d) prova-se com a definição de $\models_v$, como lá está feito.

**Exemplo resolvido — Exercício 1.8(a) e 1.8(b) (lab, proplogic.pdf) — distribuir a implicação sobre a conjunção**

**(a)** $(p \to q) \wedge (p \to r) \Leftrightarrow p \to (q \wedge r)$ — **é válida.** Prova pelas leis:

$$(p \to q) \wedge (p \to r) \Leftrightarrow (\neg p \vee q) \wedge (\neg p \vee r) \overset{\text{(distributividade)}}{\Leftrightarrow} \neg p \vee (q \wedge r) \Leftrightarrow p \to (q \wedge r)$$

**(b)** $(p \to r) \wedge (q \to r) \Leftrightarrow (p \wedge q) \to r$ — **NÃO é válida.** Contraexemplo: $v(p) = V, v(q) = F, v(r) = F$. Lado esquerdo: $p \to r = V \to F = F$; logo $(p \to r) \wedge (q \to r) = F$. Lado direito: $p \wedge q = V \wedge F = F$; logo $(p \wedge q) \to r = F \to F = V$. Lado esquerdo é $F$, lado direito é $V$ — **não são equivalentes**.

**Atenção**

Compara com atenção (a) e (b): olham parecidas (ambas "distribuem $\to$ sobre $\wedge$"), mas só a que distribui $\wedge$ do lado do **consequente** (a) é válida — distribuir sobre o **antecedente** (b) já não é. É um erro muito fácil de cometer sem verificar — usa sempre as leis ou um contraexemplo para confirmar, nunca "por analogia visual".

### Classificar uma fórmula simplificando-a (Exercício 1.9)

As leis servem também para **classificar** uma fórmula (tautologia, contradição ou satisfazível), que é o que o Exercício 1.9 pede — e o enunciado diz o método: *"You can start by simplifying the formulas using semantic equivalences"*. Não se faz a tabela de verdade.

**Definição — Método: classificar por equivalências**

1. **Eliminar as implicações** ($\phi \to \psi \Leftrightarrow \neg\phi \vee \psi$) e **empurrar as negações** para dentro (De Morgan, dupla negação).
2. **Simplificar** com as leis (complementaridade, absorção, elemento neutro/absorvente) até a fórmula ficar numa forma que se lê logo.
3. **Ler a classificação** do resultado:
   - chegou a $V \to$ **tautologia**;
   - chegou a $F \to$ **contradição**;
   - chegou a outra coisa (ex.: $p \vee \neg q$) $\to$ **satisfazível** (nem tautologia nem contradição). O enunciado pede então **uma valoração que a torna verdadeira e outra que a torna falsa**: lêem-se da forma simplificada, e **confirmam-se na fórmula original**.

**Exemplo resolvido — Exercício 1.9(d) (lab, proplogic.pdf) — tautologia**

$$\begin{aligned} & p \to (q \to (p \vee q)) & \\ \Leftrightarrow\ & \neg p \vee (\neg q \vee (p \vee q)) & \text{(implicação, 2 vezes)} \\ \Leftrightarrow\ & (\neg p \vee p) \vee (\neg q \vee q) & \text{(associatividade e comutatividade do } \vee\text{)} \\ \Leftrightarrow\ & V \vee V & \text{(complementaridade, 2 vezes)} \\ \Leftrightarrow\ & V & \text{(elemento absorvente)} \end{aligned}$$

Chegou a $V$: é **tautologia**. (Com o $\vee$ tudo ao mesmo nível, os parêntesis podem reorganizar-se à vontade — foi isso que permitiu juntar $\neg p$ com $p$ e $\neg q$ com $q$.)

**Exemplo resolvido — Exercício 1.9(j) (lab, proplogic.pdf) — contradição**

Truque: o segundo conjunto é a **negação** do primeiro.

$$\neg(p \vee \neg q) \overset{\text{(De Morgan)}}{\Leftrightarrow} \neg p \wedge \neg\neg q \overset{\text{(dupla negação)}}{\Leftrightarrow} \neg p \wedge q$$

Então, chamando $X = p \vee \neg q$:

$$(p \vee \neg q) \wedge (\neg p \wedge q) \ \Leftrightarrow\ X \wedge \neg X \overset{\text{(complementaridade)}}{\Leftrightarrow} F$$

Chegou a $F$: é **contradição**.

**Exemplo resolvido — Exercício 1.9(i) (lab, proplogic.pdf) — satisfazível**

$$\begin{aligned} & (p \vee q) \to (p \wedge q) \\ \Leftrightarrow\ & \neg(p \vee q) \vee (p \wedge q) && \text{(implicação)} \\ \Leftrightarrow\ & (\neg p \wedge \neg q) \vee (p \wedge q) && \text{(De Morgan)} \end{aligned}$$

Já não simplifica mais: não é $V$ nem $F$. Lê-se diretamente: é verdadeira quando $p$ **e** $q$ **são ambos falsos** ou **ambos verdadeiros** (é "$p \leftrightarrow q$"). Logo é **satisfazível, mas não tautologia nem contradição**.

- **Verdadeira** com $p = V, q = V$ (o segundo termo, $p \wedge q$, é $V$). Confirmação na original: $(V \vee V) \to (V \wedge V) = V \to V = V$.
- **Falsa** com $p = V, q = F$ (os dois termos têm um literal falso). Confirmação na original: $(V \vee F) \to (V \wedge F) = V \to F = F$.

**Pratica agora — Consequência e equivalência (proplogic.pdf)**

- **1.6 (b)**, **(c)** e **(d)** — provar as equivalências, pela tabela ou pelas leis (tenta pelas leis: é o que treina para o exame). Na (b), usa $\varphi \to \psi \Leftrightarrow \neg\varphi \vee \psi$ dos dois lados.
- **1.8 (c)** — é válida ou não? Mesma comparação de (a)/(b) acima: verifica com as leis de distributividade ou encontra um contraexemplo.
- **1.9 (c)** — classificar **pelo método acima** (simplificar com as leis; nada de tabela). Se não for tautologia nem contradição, dá uma valoração que a torna verdadeira e outra que a torna falsa.
- **1.9 (h)** — o mesmo, com 3 variáveis. Pista: depois de eliminar as implicações, procura uma **absorção**.

As restantes alíneas da 1.9 ((a), (b), (e), (f), (g), (k)) são a mesma simplificação repetida — não precisas de as fazer todas.

*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

## Propriedades da relação $\vDash$

As afirmações seguintes envolvem $\Gamma \vDash \theta$ de forma mais abstrata (sem fixar fórmulas concretas) — testam se percebeste bem a **definição** de consequência semântica, não cálculo. A técnica geral: para mostrar que uma afirmação é **verdadeira**, usa diretamente a definição; para mostrar que é **falsa**, basta um contraexemplo concreto (escolher $\Gamma$, $\Sigma$, $\theta$ específicos).

**Definição — Método: provar com a definição de $\vDash_v$ (formato de exame)**

Nota adicional: é assim que a professora resolve, e é o formato aceite em exame. Explicar "por palavras" não chega.

**Para mostrar $\Gamma \vDash \phi$ (ou $\vDash \phi$, com $\Gamma = \emptyset$):**

1. "Seja $v$ uma valoração tal que $\vDash_v \Gamma$." (Para $\vDash \phi$: "seja $v$ uma valoração qualquer".)
2. Abrir cada fórmula pelo seu **conectivo principal**, com a definição de $\vDash_v$ (regras 1–5 acima). Quando a regra dá um **"ou"**, **separar em casos** e fechar cada um.
3. Chegar a $\vDash_v \phi$ em todos os casos. Concluir: "como $v$ é arbitrária, $\Gamma \vDash \phi$." ■

**Alternativa: prova por absurdo.** Supõe-se $\nvDash_v \phi$ e abre-se com as regras da negação. Costuma ser a mais curta quando $\phi$ é uma implicação, porque $\nvDash_v \phi \to \psi$ força $\vDash_v \phi$ **e** $\nvDash_v \psi$. Se se chega a $\vDash_v \theta$ e $\nvDash_v \theta$, essa $v$ não existe.

**Para mostrar que é falso:** dá-se uma **valoração concreta** e verifica-se com as regras que $\vDash_v \gamma$ para cada $\gamma \in \Gamma$ e $\nvDash_v \phi$.

Regras da negação, que saem das regras 2–5:

- $\nvDash_v \phi \wedge \psi$ sse $\nvDash_v \phi$ ou $\nvDash_v \psi$
- $\nvDash_v \phi \vee \psi$ sse $\nvDash_v \phi$ e $\nvDash_v \psi$
- $\nvDash_v \phi \to \psi$ sse $\vDash_v \phi$ e $\nvDash_v \psi$

*Modus ponens* pode ser citado: se $\vDash_v \phi$ e $\vDash_v \phi \to \psi$, então $\vDash_v \psi$ (pela regra 5, $\nvDash_v \phi$ é impossível).

**Exemplo resolvido — Exercício 1.16(a), (b) e (f) (lab, proplogic.pdf) — monotonicidade**

**(a) Se $\Gamma \vDash \theta$ e $\Gamma \subseteq \Sigma$, então $\Sigma \vDash \theta$ — VERDADEIRO.**
Seja $v$ uma valoração tal que $\vDash_v \Sigma$, isto é, $\vDash_v \psi$ para todo o $\psi \in \Sigma$. Seja $\psi \in \Gamma$. Como $\Gamma \subseteq \Sigma$, $\psi \in \Sigma$, logo $\vDash_v \psi$. Portanto $\vDash_v \Gamma$. Como $\Gamma \vDash \theta$, por definição de $\vDash$ tem-se $\vDash_v \theta$. Como $v$ é arbitrária, $\Sigma \vDash \theta$. ■
(Intuição: **acrescentar premissas nunca destrói** uma consequência já válida. Chama-se **monotonicidade**.)

**(b) Se $\Sigma \vDash \theta$ e $\Gamma \subseteq \Sigma$, então $\Gamma \vDash \theta$ — FALSO** (é o recíproco de (a)).
Contraexemplo: $\Sigma = \{p,\ p \to q\}$, $\Gamma = \{p\}$, $\theta = q$. Tem-se $\Gamma \subseteq \Sigma$.

- $\Sigma \vDash q$: seja $v$ tal que $\vDash_v p$ e $\vDash_v p \to q$. Como $\vDash_v p \to q$ sse $\nvDash_v p$ ou $\vDash_v q$, e $\nvDash_v p$ é impossível, tem-se $\vDash_v q$.
- $\Gamma \nvDash q$: com $v(p) = V$, $v(q) = F$, tem-se $\vDash_v p$, isto é, $\vDash_v \Gamma$, mas $\nvDash_v q$.

As hipóteses valem e a conclusão não. **Menos premissas podem já não bastar** para a mesma conclusão.

**(f) Se $\Sigma \vDash \theta$ e $\Sigma \subseteq \Gamma$, então $\Sigma \cup \Gamma \vDash \theta$ — VERDADEIRO.** Se $\Sigma \subseteq \Gamma$, então $\Sigma \cup \Gamma = \Gamma$. A afirmação passa a ser "se $\Sigma \vDash \theta$ e $\Sigma \subseteq \Gamma$, então $\Gamma \vDash \theta$", que é **a alínea (a)** com os nomes $\Gamma$ e $\Sigma$ trocados. Já está provada. ■

**Atenção**

A lição de (a)/(b)/(f): $\Gamma \vDash \theta$ é **monótono ao acrescentar premissas** (mais hipóteses, no máximo, só ajudam a concluir mais coisas — nunca deixam de valer uma consequência já estabelecida), mas **não é monótono ao remover premissas** — retirar uma hipótese pode fazer uma conclusão válida deixar de o ser. Isto explica intuitivamente porque é que (a) e (f) são verdadeiros (só acrescentam) e (b) é falso (remove).

**Exemplo resolvido — Exercício 1.13 (lab, proplogic.pdf)**

**(a)** $\models (p \wedge \neg p) \to (p \wedge \neg r)$ — **VERDADEIRO.**
Seja $v$ uma valoração qualquer.

$$\models_v (p \wedge \neg p) \to (p \wedge \neg r) \text{ sse } \not\models_v p \wedge \neg p \text{ ou } \models_v p \wedge \neg r$$

Ora, $\models_v p \wedge \neg p$ sse $\models_v p$ e $\models_v \neg p$, isto é, sse $\models_v p$ e $\not\models_v p$, o que é impossível. Logo $\not\models_v p \wedge \neg p$ e, portanto, $\models_v (p \wedge \neg p) \to (p \wedge \neg r)$. Como $v$ é arbitrária, $\models (p \wedge \neg p) \to (p \wedge \neg r)$. ■
**(b)** $\models (q \vee \neg r) \to (q \wedge \neg q)$ — **FALSO.**
Procura-se $v$ com $\not\models_v (q \vee \neg r) \to (q \wedge \neg q)$, isto é, com $\models_v q \vee \neg r$ e $\not\models_v q \wedge \neg q$. A segunda condição vale sempre, como em (a). Basta então $\models_v q \vee \neg r$.
**Contraexemplo:** $v(q) = V$, $v(r) = V$.

- $\models_v q$, logo $\models_v q \vee \neg r$.
- $\models_v q$, logo $\not\models_v \neg q$, logo $\not\models_v q \wedge \neg q$.
- Então $\not\models_v (q \vee \neg r) \to (q \wedge \neg q)$, e a fórmula **não** é tautologia.

**(c)** $\{(p \to q) \vee r,\ ((p \to q) \vee r) \to \neg r,\ (p \to q) \to (q \to r)\} \models \neg q$ — **VERDADEIRO.**
Seja $v$ tal que $\models_v (p \to q) \vee r$, $\models_v ((p \to q) \vee r) \to \neg r$ e $\models_v (p \to q) \to (q \to r)$.

1. $\models_v ((p \to q) \vee r) \to \neg r$ sse $\not\models_v (p \to q) \vee r$ ou $\models_v \neg r$. O primeiro é impossível (é a 1.ª premissa), logo $\models_v \neg r$, isto é, $\not\models_v r$.
2. $\models_v (p \to q) \vee r$ sse $\models_v p \to q$ ou $\models_v r$. Por 1, $\not\models_v r$, logo $\models_v p \to q$.
3. $\models_v (p \to q) \to (q \to r)$ sse $\not\models_v p \to q$ ou $\models_v q \to r$. Por 2, $\models_v q \to r$.
4. $\models_v q \to r$ sse $\not\models_v q$ ou $\models_v r$. Por 1, $\not\models_v r$, logo $\not\models_v q$, isto é, $\models_v \neg q$.

Como $v$ é arbitrária, $\{(p \to q) \vee r,\ ((p \to q) \vee r) \to \neg r,\ (p \to q) \to (q \to r)\} \models \neg q$. ■
**(d)** $\{(p \vee q) \to r\} \models (p \to r) \vee (q \to r)$ — **VERDADEIRO.**
Seja $v$ tal que $\models_v (p \vee q) \to r$. Queremos $\models_v (p \to r) \vee (q \to r)$.

$$\models_v (p \vee q) \to r \text{ sse } \not\models_v p \vee q \text{ ou } \models_v r$$

- **Caso 1:** $\models_v r$**.** Como $\models_v p \to r$ sse $\not\models_v p$ ou $\models_v r$, tem-se $\models_v p \to r$. Logo $\models_v (p \to r) \vee (q \to r)$.
- **Caso 2:** $\not\models_v p \vee q$, isto é, $\not\models_v p$ e $\not\models_v q$. Como $\not\models_v p$, tem-se $\models_v p \to r$. Logo $\models_v (p \to r) \vee (q \to r)$.

Em ambos os casos $\models_v (p \to r) \vee (q \to r)$. Como $v$ é arbitrária, $\{(p \vee q) \to r\} \models (p \to r) \vee (q \to r)$. ■
**Atenção:** os dois casos estão ligados por um **"ou"**. Basta um deles acontecer. Não se pode juntar "$v(r) = V$ **e** $v(p) = F$ **e** $v(q) = F$".

**Exemplo resolvido — Exercício 1.14(a) e 1.14(d) (lab, proplogic.pdf) — sem tabela de verdade**

**(a)** $\vDash p \to (((q \to p) \to p) \to r)$ — **FALSO.**

Procura-se $v$ com $\nvDash_v$ da fórmula:

$$\nvDash_v p \to (((q \to p) \to p) \to r) \text{ sse } \vDash_v p \text{ e } \nvDash_v ((q \to p) \to p) \to r$$

$$\nvDash_v ((q \to p) \to p) \to r \text{ sse } \vDash_v (q \to p) \to p \text{ e } \nvDash_v r$$

Com $\vDash_v p$, a condição $\vDash_v (q \to p) \to p$ vale sempre, porque o consequente é satisfeito. Basta então $v(p) = V$ e $v(r) = F$.

**Contraexemplo:** $v(p) = V$, $v(q) = V$, $v(r) = F$.

- $\vDash_v p$, logo $\vDash_v (q \to p) \to p$ (consequente satisfeito).
- $\vDash_v (q \to p) \to p$ e $\nvDash_v r$, logo $\nvDash_v ((q \to p) \to p) \to r$.
- $\vDash_v p$ e $\nvDash_v ((q \to p) \to p) \to r$, logo $\nvDash_v p \to (((q \to p) \to p) \to r)$. Não é tautologia.

**(d)** $\vDash ((p \wedge q) \to (s \vee t)) \to ((p \to s) \vee (q \to t))$ — **VERDADEIRO.** Prova por absurdo.

Suponha-se que existe $v$ com $\nvDash_v$ da fórmula. Então:

1. $\vDash_v (p \wedge q) \to (s \vee t)$ e $\nvDash_v (p \to s) \vee (q \to t)$ (regra da negação de $\to$).
2. $\nvDash_v (p \to s) \vee (q \to t)$ sse $\nvDash_v p \to s$ e $\nvDash_v q \to t$, isto é: $\vDash_v p$, $\nvDash_v s$, $\vDash_v q$, $\nvDash_v t$.
3. De $\vDash_v p$ e $\vDash_v q$: $\vDash_v p \wedge q$. De $\nvDash_v s$ e $\nvDash_v t$: $\nvDash_v s \vee t$.
4. Então $\nvDash_v (p \wedge q) \to (s \vee t)$, o que contradiz 1.

Logo não existe tal $v$: a fórmula é satisfeita por todas as valorações, $\vDash ((p \wedge q) \to (s \vee t)) \to ((p \to s) \vee (q \to t))$. ■

**Atenção**

Nota adicional sobre "sem tabela de verdade": nos dois casos começa-se por **supor $\nvDash_v$ da fórmula** e abrir com as regras da negação. Se se chega a uma valoração consistente, como em (a), essa valoração **é** o contraexemplo, e verifica-se regra a regra. Se se chega a $\vDash_v \theta$ e $\nvDash_v \theta$, como em (d), é uma prova por absurdo de que a fórmula é tautologia. (Na 1.9, pelo contrário, o enunciado manda simplificar pelas leis.)

**Pratica agora — Propriedades de $\vDash$ (proplogic.pdf)**

- **1.15** — mesma ideia do Exercício 1.16 acima, mas com $\vDash$ e $\Leftrightarrow$ entre fórmulas genéricas $\varphi, \psi, \theta, \gamma$: se for verdadeiro, prova com a definição de $\vDash_v$ (método acima); se for falso, instancia com fórmulas concretas ($\varphi = p$, …) e dá a valoração do contraexemplo, verificada regra a regra.

*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

## Quantas proposições podem ser simultaneamente verdadeiras

**Exemplo resolvido — Exercício 1.7 (lab, proplogic.pdf)**

Atribuindo valores de verdade a $p, q, r$, quantas das cinco disjunções $p \vee \neg q$, $\neg p \vee q$, $q \vee r$, $q \vee \neg r$, $\neg q \vee \neg r$ podem ser simultaneamente verdadeiras?
Como há só 3 variáveis, é viável construir a tabela completa (8 linhas) e contar, em cada linha, quantas das 5 fórmulas são $V$:

| $p$ | $q$ | $r$ | $p \vee \neg q$ | $\neg p \vee q$ | $q \vee r$ | $q \vee \neg r$ | $\neg q \vee \neg r$ | nº de V |
|---|---|---|---|---|---|---|---|---|
| V | V | V | V | V | V | V | F | 4 |
| V | V | F | V | V | V | V | V | **5** |
| V | F | V | V | F | V | F | V | 3 |
| V | F | F | V | F | F | V | V | 3 |
| F | V | V | F | V | V | V | F | 3 |
| F | V | F | F | V | V | V | V | 4 |
| F | F | V | V | V | V | F | V | 4 |
| F | F | F | V | V | F | V | V | 4 |

O **máximo** é **5** (todas simultaneamente verdadeiras), atingido em $p = V, q = V, r = F$ — por exemplo, verifica-se que $\neg q \vee \neg r$ dá $F \vee V = V$ porque $r = F$. O **mínimo** ao longo de todas as valorações é **3** (nunca é possível ter menos de 3 verdadeiras ao mesmo tempo, e isso acontece em três linhas diferentes da tabela).

## Conjuntos completos de conectivos

**Definição — Conjunto completo de conectivos**

Um conjunto de conectivos $C$ é **completo** se, para qualquer função de verdade $f : \{V, F\}^n \to \{V, F\}$, existe uma fórmula $\phi$ com $n$ variáveis, usando só conectivos de $C$, tal que $\phi$ **induz** exatamente $f$ (isto é, $F_\phi = f$, onde $F_\phi(x_1, \ldots, x_n) = v_\chi(\phi)$ para $v_\chi(p_i) = x_i$). Por outras palavras: $C$ chega para exprimir *qualquer* função booleana possível.

**Prova de que $\{\wedge, \vee, \neg\}$ é completo** (dos slides, por indução em $n$, o número de variáveis):

- **Base ($n = 1$):** há exatamente 4 funções de verdade de aridade 1 (uma tabela com 2 linhas tem $2^2 = 4$ combinações possíveis de saída). Para cada uma, exibe-se uma fórmula que a realiza: \$f_1 = \$ "sempre F" $\to \phi_1 = p \wedge \neg p$; $f_2 = \neg p \to \phi_2 = \neg p$; $f_3 = p \to \phi_3 = p$; $f_4 =$ "sempre V" $\to \phi_4 = p \vee \neg p$.
- **Passo indutivo:** assumindo que a propriedade vale para $n$, seja $f : \{V, F\}^{n+1} \to \{V, F\}$. Define-se $f_1(x_1, \ldots, x_n) = f(x_1, \ldots, x_n, V)$ e $f_2(x_1, \ldots, x_n) = f(x_1, \ldots, x_n, F)$ (fixando a última variável). Por hipótese de indução existem $\phi_1, \phi_2$ com $F_{\phi_i} = f_i$. Toma-se $\phi = (p_{n+1} \wedge \phi_1) \vee (\neg p_{n+1} \wedge \phi_2)$: quando $x_{n+1} = V$, $\phi$ reduz-se a $\phi_1$; quando $x_{n+1} = F$, reduz-se a $\phi_2$ — exatamente o comportamento de $f$.

Como consequência imediata (usando as leis de De Morgan e a lei da implicação para reescrever $\wedge$ e $\vee$), também $\{\neg, \to\}$ é completo: $\phi \wedge \psi \Leftrightarrow \neg(\phi \to \neg\psi)$ e $\phi \vee \psi \Leftrightarrow \neg\phi \to \psi$.

(Na Aula 3 aparece uma segunda prova, mais direta, do mesmo resultado: a construção da **forma normal disjuntiva** a partir da tabela de verdade.)

**Exemplo resolvido — Exercício 1.19(a) (lab, proplogic.pdf) — negação, disjunção é completo**

Mostrar que $p \to q$, $p \leftrightarrow q$ e $p \wedge q$ se escrevem só com $\neg$ e $\vee$ (a mesma técnica da prova acima, mas explicitada concretamente):

- $p \to q \Leftrightarrow \neg p \vee q$ (é diretamente a lei da implicação).
- $p \wedge q \Leftrightarrow \neg(\neg p \vee \neg q)$ (De Morgan aplicado ao contrário: $\neg\neg(p \wedge q) \Leftrightarrow \neg(\neg p \vee \neg q)$, e $\neg\neg(p \wedge q) \Leftrightarrow p \wedge q$).
- $p \leftrightarrow q \Leftrightarrow (p \to q) \wedge (q \to p)$. Usando os dois pontos anteriores para reescrever cada peça só com $\neg, \vee$: $p \to q \Leftrightarrow \neg p \vee q$ e $q \to p \Leftrightarrow \neg q \vee p$; e a conjunção dessas duas, pelo ponto anterior ($\phi \wedge \psi \Leftrightarrow \neg(\neg\phi \vee \neg\psi)$), fica:

$$p \leftrightarrow q \;\Leftrightarrow\; \neg\big(\neg(\neg p \vee q) \vee \neg(\neg q \vee p)\big)$$

— uma fórmula só com $\neg$ e $\vee$, como pedido.

**Exemplo resolvido — Exercício 1.19(b) (lab, proplogic.pdf) — NAND é universal**

Define-se um novo conectivo $\bar{\wedge}$ ("NAND") pela tabela: $V \bar{\wedge} V = F$; $V \bar{\wedge} F = F \bar{\wedge} V = F \bar{\wedge} F = V$ (ou seja, $p \bar{\wedge} q \Leftrightarrow \neg(p \wedge q)$: só é falso quando ambos são verdadeiros). Mostrar que $\neg p$, $p \wedge q$ e $p \vee q$ se escrevem só com $\bar{\wedge}$:
**Negação:** $p \bar{\wedge} p \Leftrightarrow \neg(p \wedge p) \Leftrightarrow \neg p$ (idempotência). Logo $\neg p = p \bar{\wedge} p$.
**Conjunção:** $p \bar{\wedge} q \Leftrightarrow \neg(p \wedge q)$, logo $\neg(p \bar{\wedge} q) \Leftrightarrow p \wedge q$. E $\neg X = X \bar{\wedge} X$ (pelo caso anterior, com $X = p \bar{\wedge} q$), logo:

$$p \wedge q \;=\; (p \bar{\wedge} q) \;\bar{\wedge}\; (p \bar{\wedge} q)$$

**Disjunção:** por De Morgan, $p \vee q \Leftrightarrow \neg(\neg p \wedge \neg q) \Leftrightarrow \neg p \bar{\wedge} \neg q$ (a própria definição de $\bar{\wedge}$ já é a negação de um $\wedge$). Substituindo $\neg p = p \bar{\wedge} p$ e $\neg q = q \bar{\wedge} q$:

$$p \vee q \;=\; (p \bar{\wedge} p) \;\bar{\wedge}\; (q \bar{\wedge} q)$$

Como $\{\neg, \wedge, \vee\}$ já é completo e todos os três se escrevem só com $\bar{\wedge}$, conclui-se que $\{\bar{\wedge}\}$ **sozinho já é um conjunto completo de conectivos** — é o princípio por trás dos circuitos NAND em eletrónica digital (qualquer circuito lógico pode ser construído só com portas NAND).

## Aplicação: modelar problemas em lógica proposicional

Os problemas seguintes não introduzem teoria nova — mostram como usar tudo o que já foi visto (tradução para fórmulas, tabelas de verdade, satisfazibilidade de conjuntos) para resolver problemas de raciocínio "do mundo real". A técnica chave nos problemas de cavaleiros/vilões é a **auto-referência**: se $a$ representa "A é cavaleiro" (diz sempre a verdade) e $S$ é a fórmula que traduz a frase que A disse, então tem de valer $a \leftrightarrow S$ — A é cavaleiro *sse* o que disse é verdade.

**Exemplo resolvido — Exercício 1.10(a) (lab, proplogic.pdf) — ilha dos cavaleiros e vilões**

A diz: "Pelo menos um de nós [A, B] é vilão." O que são A e B?
Seja $a$ = "A é cavaleiro", $b$ = "B é cavaleiro". A frase de A traduz-se por $S = \neg a \vee \neg b$. A condição de auto-referência é $a \leftrightarrow S$, isto é $a \leftrightarrow (\neg a \vee \neg b)$. Testam-se as 4 valorações de $(a, b)$:

| $a$ | $b$ | $S = \neg a \vee \neg b$ | $a \leftrightarrow S$ |
|---|---|---|---|
| V | V | F | F |
| V | F | V | **V** |
| F | V | V | F |
| F | F | V | F |

Só a linha $a = V, b = F$ é consistente (só nessa a biconditional dá $V$; nas outras a suposição contradiz-se a si própria). **A é cavaleiro, B é vilão.**

**Exemplo resolvido — Exercício 1.10(e) (lab, proplogic.pdf) — três pessoas**

A, B, C. A diz: "Somos todos vilões." B diz: "Exatamente um de nós é cavaleiro." O que são A, B e C?
Sejam $a, b, c$ os indicadores "é cavaleiro" de A, B, C. Traduzindo: $S_A = \neg a \wedge \neg b \wedge \neg c$ (todos vilões) e $S_B = (b \wedge \neg c) \vee (\neg b \wedge c)$ (exatamente um entre B e C é cavaleiro — note-se que $S_B$ já não depende de $a$ diretamente, mas a condição "exatamente um dos três" só se decompõe assim depois de se saber o valor de $a$; para manter a tabela simples, calcula-se $S_B$ como "exatamente um de $\{a, b, c\}$" diretamente nas 8 linhas).
Constrói-se a tabela com as duas condições de auto-referência, $\text{cond}_1 = (a \leftrightarrow S_A)$ e $\text{cond}_2 = (b \leftrightarrow S_B)$, para as 8 valorações de $(a, b, c)$ — só interessam as linhas onde **ambas** são $V$:

| $a$ | $b$ | $c$ | $S_A$ | $\text{cond}_1$ | $S_B$ (exat. 1 de a,b,c) | $\text{cond}_2$ |
|---|---|---|---|---|---|---|
| V | V | V | F | F | F | F |
| V | V | F | F | F | F | F |
| V | F | V | F | F | F | V |
| V | F | F | F | F | V | F |
| F | V | V | F | V | F | F |
| F | V | F | F | V | **V** | **V** |
| F | F | V | F | V | F | F |
| F | F | F | V | F | V | F |

Só a linha $a = F, b = V, c = F$ tem $\text{cond}_1 = V$ **e** $\text{cond}_2 = V$ simultaneamente. **A é vilão, B é cavaleiro, C é vilão** — e é a única solução consistente com as duas afirmações.

**Exemplo resolvido — Exercício 1.11 (lab, proplogic.pdf) — suspeitos de um crime**

Três suspeitos, A, B, C, fazem declarações: A: "B é culpado, mas C é inocente." B: "Se A é culpado, então C é culpado." C: "Eu sou inocente, mas um dos outros dois é culpado."
Sejam $g_a, g_b, g_c$ = "A/B/C é culpado". Traduzindo cada declaração:

$$S_A = g_b \wedge \neg g_c \qquad S_B = g_a \to g_c \qquad S_C = \neg g_c \wedge (g_a \vee g_b)$$

Constrói-se **uma única tabela de verdade** com as 8 combinações de $(g_a, g_b, g_c)$ e as três colunas $S_A, S_B, S_C$ — e todas as alíneas seguintes respondem-se lendo linhas/colunas desta mesma tabela, sem repetir cálculos:

| $g_a$ | $g_b$ | $g_c$ | $S_A = g_b \wedge \neg g_c$ | $S_B = g_a \to g_c$ | $S_C = \neg g_c \wedge (g_a \vee g_b)$ |
|---|---|---|---|---|---|
| V | V | V | F | V | F |
| V | V | F | **V** | F | **V** |
| V | F | V | F | V | F |
| V | F | F | F | F | **V** |
| F | V | V | F | V | F |
| F | V | F | **V** | **V** | **V** |
| F | F | V | F | V | F |
| F | F | F | F | V | F |

**(a) As três declarações são compatíveis?** Pergunta-se se $\{S_A, S_B, S_C\}$ é satisfazível. Com $v(g_a) = F$, $v(g_b) = V$, $v(g_c) = F$: $\vDash_v S_A$ ($\vDash_v g_b$, $\nvDash_v g_c$), $\vDash_v S_B$ ($\nvDash_v g_a$) e $\vDash_v S_C$ ($\nvDash_v g_c$, $\vDash_v g_b$). **Sim, são compatíveis**, e pela tabela essa é a **única** valoração que as satisfaz (só B culpado).

**Exemplo resolvido — Exercício 1.11 (cont.) — consequência**

**(b) Alguma declaração é consequência das outras duas?** Há três consequências para testar.
$\{S_A, S_B\} \vDash S_C$*: sim.* Seja $v$ tal que $\vDash_v S_A$ e $\vDash_v S_B$.

- $\vDash_v g_b \wedge \neg g_c$ sse $\vDash_v g_b$ e $\nvDash_v g_c$.
- $\vDash_v g_a \to g_c$ sse $\nvDash_v g_a$ ou $\vDash_v g_c$. Como $\nvDash_v g_c$, tem-se $\nvDash_v g_a$.
- Então $\vDash_v \neg g_c$ e, como $\vDash_v g_b$, $\vDash_v g_a \vee g_b$. Logo $\vDash_v \neg g_c \wedge (g_a \vee g_b)$, isto é, $\vDash_v S_C$. ■

$\{S_B, S_C\} \vDash S_A$*: sim.* Seja $v$ tal que $\vDash_v S_B$ e $\vDash_v S_C$.

- $\vDash_v \neg g_c \wedge (g_a \vee g_b)$ sse $\nvDash_v g_c$ e $\vDash_v g_a \vee g_b$.
- $\vDash_v g_a \to g_c$ sse $\nvDash_v g_a$ ou $\vDash_v g_c$. Como $\nvDash_v g_c$, tem-se $\nvDash_v g_a$.
- $\vDash_v g_a \vee g_b$ sse $\vDash_v g_a$ ou $\vDash_v g_b$. Como $\nvDash_v g_a$, tem-se $\vDash_v g_b$.
- Logo $\vDash_v g_b \wedge \neg g_c$, isto é, $\vDash_v S_A$. ■

$\{S_A, S_C\} \nvDash S_B$. Contraexemplo: $v(g_a) = V$, $v(g_b) = V$, $v(g_c) = F$ (2.ª linha da tabela).

- $\vDash_v g_b$ e $\nvDash_v g_c$, logo $\vDash_v S_A$.
- $\nvDash_v g_c$ e $\vDash_v g_a$, logo $\vDash_v S_C$.
- $\vDash_v g_a$ e $\nvDash_v g_c$, logo $\nvDash_v g_a \to g_c$, isto é, $\nvDash_v S_B$.

**Exemplo resolvido — Exercício 1.11 (cont.) — hipóteses sobre quem diz a verdade**

**(c) Assumindo que os três são inocentes, quem mentiu?** Linha $g_a = F, g_b = F, g_c = F$: $S_A = F$ (A mentiu), $S_B = V$ (B disse a verdade), $S_C = F$ (C mentiu). **A e C mentiram; B disse a verdade.**
**(d) Assumindo que os três disseram a verdade, quem é inocente/culpado?** Precisa de $S_A = S_B = S_C = V$ — é exatamente a linha de (a): $g_a = F, g_b = V, g_c = F$. **Só B é culpado; A e C são inocentes.**

**Exemplo resolvido — Exercício 1.11 (cont.) — inocentes dizem a verdade**

**(e) Assumindo que o inocente disse a verdade e o culpado mentiu, quem é inocente/culpado?** Para cada pessoa $X$: $X$ é inocente sse a sua declaração é verdadeira. Isso dá o conjunto

$$\Delta = \{\, S_A \leftrightarrow \neg g_a,\quad S_B \leftrightarrow \neg g_b,\quad S_C \leftrightarrow \neg g_c \,\}$$

e procuram-se as $v$ com $\vDash_v \Delta$. ($\vDash_v \phi \leftrightarrow \psi$ sse $\vDash_v \phi$ e $\vDash_v \psi$ têm o mesmo valor: ou ambos valem ou nenhum vale.)
Seja $v$ tal que $\vDash_v \Delta$.

1. $\vDash_v g_a$**.** Suponha-se $\nvDash_v g_a$. Então $\vDash_v \neg g_a$, logo $\vDash_v S_A$: $\vDash_v g_b$ e $\nvDash_v g_c$. Como $\vDash_v g_b$, $\nvDash_v \neg g_b$, logo $\nvDash_v S_B$, isto é, $\vDash_v g_a$ e $\nvDash_v g_c$. Mas supôs-se $\nvDash_v g_a$. Contradição.
2. $\vDash_v g_c$**.** Por 1, $\nvDash_v \neg g_a$, logo $\nvDash_v S_A$: $\nvDash_v g_b$ ou $\vDash_v g_c$. Suponha-se $\nvDash_v g_c$. Então $\nvDash_v g_b$, logo $\vDash_v \neg g_b$ e $\vDash_v S_B$: $\nvDash_v g_a$ ou $\vDash_v g_c$. Nenhuma das duas vale (por 1 e pela suposição). Contradição.
3. $\nvDash_v g_b$**.** Suponha-se $\vDash_v g_b$. Então $\nvDash_v \neg g_b$, logo $\nvDash_v S_B$: $\vDash_v g_a$ e $\nvDash_v g_c$. Isto contradiz 2.

Logo a única hipótese é $v(g_a) = V$, $v(g_b) = F$, $v(g_c) = V$. Verificação de que $\vDash_v \Delta$:

- $\nvDash_v S_A$ (porque $\vDash_v g_c$) e $\nvDash_v \neg g_a$. ✓
- $\vDash_v S_B$ (porque $\vDash_v g_c$) e $\vDash_v \neg g_b$. ✓
- $\nvDash_v S_C$ (porque $\vDash_v g_c$) e $\nvDash_v \neg g_c$. ✓

**A e C são culpados; B é inocente.** (Na tabela, é a única linha onde $S_A$, $S_B$ e $S_C$ coincidem com $\neg g_a$, $\neg g_b$ e $\neg g_c$.)

**Atenção**

Repara como (c), (d) e (e) dão **três respostas diferentes** ao mesmo enunciado — cada uma corresponde a uma hipótese diferente sobre quem diz a verdade. É um erro comum tentar decidir "quem é mesmo culpado" sem antes fixar qual das três hipóteses está a ser assumida; a tabela de verdade única (com as colunas $S_A, S_B, S_C$) é o que permite responder às três sem ambiguidade nem trabalho repetido.

**Pratica agora — Modelação (proplogic.pdf)**

- **1.10 (g)** — raciocínio indireto diferente: não analisas o que A/B disseram, mas o que **C disse**.
- **1.10 (j)** — aqui és tu que **constróis** a pergunta certa (pista do enunciado: relaciona, por equivalência, o que queres saber com o tipo do habitante).
- **1.12** — mesma técnica de modelação do Exercício 1.11 (uma variável por cadeira, uma fórmula por regra).
- *Opcional:* **1.10 (k)**, que o professor chama "um problema invulgar".

As restantes alíneas de 1.10 ((b), (c), (d), (f), (h), (i), (l)) repetem a técnica de (a)/(e) — não precisas de as fazer.
*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

# Aula 3 — Formas Normais e Satisfazibilidade

Até aqui, para saber se uma fórmula é satisfazível ou tautologia, a única ferramenta geral era a tabela de verdade — com $2^n$ linhas para $n$ variáveis. Esta aula mostra que qualquer fórmula pode ser **transformada numa fórmula semanticamente equivalente com uma forma especial** (uma *forma normal*), e que, para algumas dessas formas, decidir a satisfazibilidade ou a validade passa a ser trivial (basta "olhar" para a fórmula). Algumas formas normais existem para **qualquer** fórmula (negativa, disjuntiva, conjuntiva); outras só existem para certas classes de fórmulas (Horn).

## Literais e forma normal negativa (FNN)

**Definição — Literal e forma normal negativa**

Um **literal** é uma variável proposicional $p$ ou a sua negação $\neg p$. ($p$ diz-se literal **positivo**, $\neg p$ literal **negativo**.)
Uma fórmula está em **forma normal negativa** se a negação $\neg$ só aparece aplicada diretamente a variáveis, isto é, só dentro de literais. Ex: $(\neg p \wedge \neg q) \vee p$ está em FNN; $\neg(p \vee q)$ não está.

**Definição — Proposição (existência da FNN)**

Qualquer fórmula que só use os conectivos $\wedge$, $\vee$ e $\neg$ é semanticamente equivalente a uma fórmula em forma normal negativa.
**Prova:** basta aplicar repetidamente as **leis de De Morgan** (que "empurram" cada $\neg$ para dentro de um $\wedge/\vee$, trocando-o pelo outro) e eliminar as **duplas negações** ($\neg\neg\phi \Leftrightarrow \phi$) até cada $\neg$ ficar encostado a uma variável.

**Exemplo resolvido — FNN de $\neg((p \vee q) \wedge \neg p)$ (slides)**

$$\begin{aligned} & \neg((p \vee q) \wedge \neg p) \\ \overset{\text{(De Morgan, no } \wedge \text{ exterior)}}{\Leftrightarrow} \quad & \neg(p \vee q) \vee \neg\neg p \\ \overset{\text{(De Morgan, em } \neg(p\vee q))}{\Leftrightarrow} \quad & (\neg p \wedge \neg q) \vee \neg\neg p \\ \overset{\text{(dupla negação)}}{\Leftrightarrow} \quad & (\neg p \wedge \neg q) \vee p \end{aligned}$$

Em cada passo o $\neg$ desce um nível na árvore sintática; quando todos os $\neg$ estão encostados a variáveis, parou-se: está em FNN.

**Atenção**

Nota adicional (não estava explícito nos slides): a proposição só fala de fórmulas com $\wedge, \vee, \neg$. Se a fórmula tiver $\rightarrow$ (ou $\leftrightarrow$), **elimina-os primeiro**: $\phi \rightarrow \psi \Leftrightarrow \neg\phi \vee \psi$ e $\phi \leftrightarrow \psi \Leftrightarrow (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi)$. Só depois se empurram as negações. Esquecer este passo é o erro mais comum — $\neg(p \rightarrow q)$ **não** é "$\neg p \rightarrow \neg q$"; é $p \wedge \neg q$ (ver Exercício 1.6(a) na Aula 2).

## Forma normal disjuntiva (FND)

**Definição — Forma normal disjuntiva**

Uma fórmula está em **forma normal disjuntiva** se é uma **disjunção de conjunções de literais**:

$$(\alpha_{11} \wedge ... \wedge \alpha_{1k_1}) \vee ... \vee (\alpha_{n1} \wedge ... \wedge \alpha_{nk_n})$$

onde cada $\alpha_{ij}$ é um literal. ("Um *ou* de *e*'s".)
Exemplos: $p \wedge q$ (uma só conjunção); $p \vee \neg p$ (duas conjunções, cada uma com um só literal); $(p \wedge q \wedge \neg r) \vee (\neg p \wedge r)$.

Há duas formas de obter uma FND: **a partir da tabela de verdade** (lema seguinte) ou **a partir da fórmula**, por equivalências.

### FND a partir da tabela de verdade

**Definição — Lema (toda a função de verdade tem uma FND)**

Para qualquer função $f : \{V, F\}^n \to \{V, F\}$ existe uma fórmula $\phi$ em FND, com $n$ variáveis, tal que $F_\phi = f$.
**Prova (construtiva — é o próprio método):**

- Se $f$ dá $F$ em todas as linhas, toma-se $\phi = p_1 \wedge \neg p_1$ (sempre falsa).
- Senão, para cada linha (valoração) $v$ da tabela, define-se o literal $l_i^v = p_i$ se $v(p_i) = V$ e $l_i^v = \neg p_i$ se $v(p_i) = F$, e a conjunção $\phi_v = l_1^v \wedge ... \wedge l_n^v$.
- **Porquê isto funciona:** cada $l_i^v$ é, por construção, verdadeiro em $v$, logo $v(\phi_v) = V$. E em qualquer **outra** linha $v'$, pelo menos uma variável tem valor diferente, o literal correspondente é falso, e $\phi_v$ é falsa. Ou seja: $\phi_v$ **é verdadeira exatamente na linha** $v$ **e em mais nenhuma.**
- Toma-se $\phi = \bigvee_{f(v)=V} \phi_v$: a disjunção das $\phi_v$ das linhas em que $f$ dá $V$. Então $\phi$ é verdadeira exatamente nessas linhas — ou seja, $F_\phi = f$.

**Exemplo resolvido — FND a partir de uma tabela (slides)**

| linha | $x_1$ | $x_2$ | $x_3$ | $f$ | conjunção $\phi_v$ (só para as linhas com $V$) |
|---|---|---|---|---|---|
| 1 | V | V | V | V | $p_1 \wedge p_2 \wedge p_3$ |
| 2 | V | V | F | V | $p_1 \wedge p_2 \wedge \neg p_3$ |
| 3 | V | F | V | V | $p_1 \wedge \neg p_2 \wedge p_3$ |
| 4 | V | F | F | F | — |
| 5 | F | V | V | V | $\neg p_1 \wedge p_2 \wedge p_3$ |
| 6 | F | V | F | F | — |
| 7 | F | F | V | F | — |
| 8 | F | F | F | F | — |

Regra para escrever cada conjunção: variável a $V$ na linha → entra **positiva**; variável a $F$ → entra **negada**. Ex.: linha 3 tem $x_1 = V, x_2 = F, x_3 = V$, logo $p_1 \wedge \neg p_2 \wedge p_3$.
Juntando com $\vee$ as 4 conjunções das linhas com $V$:

$$(p_1 \wedge p_2 \wedge p_3) \vee (p_1 \wedge p_2 \wedge \neg p_3) \vee (p_1 \wedge \neg p_2 \wedge p_3) \vee (\neg p_1 \wedge p_2 \wedge p_3)$$

(Repara: $f$ é a função "maioria" — é $V$ quando pelo menos duas das três variáveis são $V$.)

**Atenção**

Este lema é **outra prova** de que $\{\wedge, \vee, \neg\}$ é um conjunto completo de conectivos (Aula 2, "Conjuntos completos de conectivos"): a fórmula construída só usa $\wedge$, $\vee$ e $\neg$ e realiza qualquer $f$. Lá a prova era por indução no número de variáveis; aqui é direta, linha a linha. Ambas chegam ao mesmo resultado.

### FND a partir de uma fórmula

**Definição — Corolário (toda a fórmula tem uma FND equivalente)**

Qualquer fórmula é semanticamente equivalente a uma fórmula em FND. Para a obter:

1. eliminar $\rightarrow$ e $\leftrightarrow$, ficando só com $\wedge$, $\vee$, $\neg$;
2. passar para **forma normal negativa** (De Morgan + dupla negação);
3. aplicar a **distributividade do $\wedge$ sobre o $\vee$** até não haver nenhum $\vee$ dentro de um $\wedge$:

$$(\phi \vee \psi) \wedge \theta \Leftrightarrow (\phi \wedge \theta) \vee (\psi \wedge \theta) \qquad \theta \wedge (\phi \vee \psi) \Leftrightarrow (\theta \wedge \phi) \vee (\theta \wedge \psi)$$

**Exemplo resolvido — FND de $(p \vee r) \leftrightarrow (q \wedge \neg p)$, parte 1: passos 1 e 2 (slides)**

**Passo 1 — eliminar $\leftrightarrow$ e $\rightarrow$:**

$$\begin{aligned} & (p \vee r) \leftrightarrow (q \wedge \neg p) \\ \Leftrightarrow\ & ((p \vee r) \rightarrow (q \wedge \neg p)) \wedge ((q \wedge \neg p) \rightarrow (p \vee r)) && (\leftrightarrow \text{ como duas implicações}) \\ \Leftrightarrow\ & (\neg(p \vee r) \vee (q \wedge \neg p)) \wedge (\neg(q \wedge \neg p) \vee (p \vee r)) && (\text{implicação, 2 vezes}) \end{aligned}$$

**Passo 2 — forma normal negativa.** No lado esquerdo, $\neg(p \vee r) \Leftrightarrow \neg p \wedge \neg r$ (De Morgan). No lado direito, $\neg(q \wedge \neg p) \Leftrightarrow \neg q \vee \neg\neg p \Leftrightarrow \neg q \vee p$ (De Morgan + dupla negação). Fica:

$$\underbrace{((\neg p \wedge \neg r) \vee (q \wedge \neg p))}_{X} \wedge \underbrace{((\neg q \vee p) \vee (p \vee r))}_{A \vee B}$$

com $A = \neg q \vee p$ e $B = p \vee r$. Isto já está em FNN, mas **não** em FND: há $\vee$ dentro do $\wedge$ principal.

**Exemplo resolvido — FND de $(p \vee r) \leftrightarrow (q \wedge \neg p)$, parte 2: distributividade (slides)**

**Passo 3a** — $X \wedge (A \vee B) \Leftrightarrow (X \wedge A) \vee (X \wedge B)$:

$$(((\neg p \wedge \neg r) \vee (q \wedge \neg p)) \wedge A) \vee (((\neg p \wedge \neg r) \vee (q \wedge \neg p)) \wedge B)$$

**Passo 3b** — em cada um dos dois blocos, $X = C_1 \vee C_2$ (com $C_1 = \neg p \wedge \neg r$, $C_2 = q \wedge \neg p$) volta a distribuir: $(C_1 \vee C_2) \wedge A \Leftrightarrow (C_1 \wedge A) \vee (C_2 \wedge A)$, e o mesmo com $B$:

$$(\neg p \wedge \neg r \wedge (\neg q \vee p)) \vee (q \wedge \neg p \wedge (\neg q \vee p)) \vee (\neg p \wedge \neg r \wedge (p \vee r)) \vee (q \wedge \neg p \wedge (p \vee r))$$

**Passo 3c** — cada um dos 4 termos ainda tem um $\vee$ lá dentro; distribui-se cada um $(K \wedge (a \vee b) \Leftrightarrow (K \wedge a) \vee (K \wedge b))$:

- **Termo 1** — $K = \neg p \wedge \neg r$, $(a \vee b) = \neg q \vee p$: $(\neg p \wedge \neg r \wedge \neg q) \vee (\neg p \wedge \neg r \wedge p)$
- **Termo 2** — $K = q \wedge \neg p$, $(a \vee b) = \neg q \vee p$: $(q \wedge \neg p \wedge \neg q) \vee (q \wedge \neg p \wedge p)$
- **Termo 3** — $K = \neg p \wedge \neg r$, $(a \vee b) = p \vee r$: $(\neg p \wedge \neg r \wedge p) \vee (\neg p \wedge \neg r \wedge r)$
- **Termo 4** — $K = q \wedge \neg p$, $(a \vee b) = p \vee r$: $(q \wedge \neg p \wedge p) \vee (q \wedge \neg p \wedge r)$

A FND é a disjunção destas 8 conjunções:

$$(\neg p \wedge \neg r \wedge \neg q) \vee (\neg p \wedge \neg r \wedge p) \vee (q \wedge \neg p \wedge \neg q) \vee (q \wedge \neg p \wedge p) \vee (\neg p \wedge \neg r \wedge p) \vee (\neg p \wedge \neg r \wedge r) \vee (q \wedge \neg p \wedge p) \vee (q \wedge \neg p \wedge r)$$

## Satisfazibilidade de uma FND

**Definição — Lema e corolário**

**Lema.** Uma conjunção de literais $l_1 \wedge ... \wedge l_n$ é satisfazível **sse** não contém um par complementar, isto é, para todos os $i, j$, $l_i$ não é $\neg l_j$. (Se não há par $p/\neg p$, basta pôr cada literal positivo a $V$ e cada negativo a $F$ — não há conflito. Se há $p$ e $\neg p$, nunca podem ser ambos verdadeiros.)
**Corolário.** Uma fórmula em FND é satisfazível **sse alguma** das suas conjunções é satisfazível (uma disjunção é verdadeira se **um** dos termos for).

**Exemplo resolvido**

- $p \wedge \neg q \wedge \neg r \wedge q$: contém $\neg q$ e $q$ → **não** é satisfazível.
- $\neg p \wedge q \wedge \neg r \wedge \neg s$: nenhum par complementar → **é** satisfazível (com $p = F$, $q = V$, $r = F$, $s = F$).

**Exemplo resolvido — $(p \vee r) \leftrightarrow (q \wedge \neg p)$ é satisfazível? (slides)**

Usa-se a FND obtida acima e verifica-se **cada** uma das 8 conjunções:

| # | conjunção | par complementar? | satisfazível? |
|---|---|---|---|
| 1 | $\neg p \wedge \neg r \wedge \neg q$ | nenhum | **sim** |
| 2 | $\neg p \wedge \neg r \wedge p$ | $\neg p$, $p$ | não |
| 3 | $q \wedge \neg p \wedge \neg q$ | $q$, $\neg q$ | não |
| 4 | $q \wedge \neg p \wedge p$ | $\neg p$, $p$ | não |
| 5 | $\neg p \wedge \neg r \wedge p$ | $\neg p$, $p$ | não |
| 6 | $\neg p \wedge \neg r \wedge r$ | $\neg r$, $r$ | não |
| 7 | $q \wedge \neg p \wedge p$ | $\neg p$, $p$ | não |
| 8 | $q \wedge \neg p \wedge r$ | nenhum | **sim** |

Há (pelo menos) uma conjunção satisfazível → a fórmula **é satisfazível**. A conjunção 1 dá a valoração $p = q = r = F$; a 8 dá $p = F$, $q = r = V$.
Como as conjunções insatisfazíveis são sempre falsas, podem ser **apagadas** da disjunção sem mudar nada ($\phi \vee F \Leftrightarrow \phi$), e fica a FND simplificada:

$$(p \vee r) \leftrightarrow (q \wedge \neg p) \;\Leftrightarrow\; (\neg p \wedge \neg q \wedge \neg r) \vee (\neg p \wedge q \wedge r)$$

**Verificação direta** (para confirmar que não houve engano): se $p = V$, o lado esquerdo $p \vee r$ é $V$ e o direito $q \wedge \neg p$ é $F$ → a equivalência é $F$. Se $p = F$, fica $r \leftrightarrow q$, que é $V$ só para $q = r = F$ ou $q = r = V$. São exatamente as duas conjunções que sobraram. ✓

**Atenção**

Nota adicional (dos apontamentos da Prof.ª Nelma Moreira, em `Teoricas/`): se a fórmula **já está** em FND, este teste é **linear** no tamanho da fórmula — muito melhor do que os $2^n$ da tabela de verdade. Mas **atenção**: *obter* a FND pode ser caro — cada distributividade duplica parte da fórmula (o exemplo acima passou de uma fórmula pequena para 8 conjunções), e no pior caso a FND tem tamanho exponencial. Não se conhece nenhum algoritmo polinomial para a satisfazibilidade de uma fórmula qualquer (problema **SAT**); encontrar um responderia à pergunta "P = NP?". O que há são classes de fórmulas em que o problema é fácil (FND, Horn) e algoritmos que na prática funcionam bem para fórmulas em FNC.

## Forma normal conjuntiva (FNC)

**Definição — Forma normal conjuntiva**

Uma fórmula está em **forma normal conjuntiva** se é uma **conjunção de disjunções de literais** ("um *e* de *ou*'s"):

$$(\alpha_{11} \vee ... \vee \alpha_{1k_1}) \wedge ... \wedge (\alpha_{n1} \vee ... \vee \alpha_{nk_n})$$

Cada disjunção de literais chama-se uma **cláusula**. (Na Aula 4 as cláusulas passam a escrever-se como **conjuntos de literais**; ver a secção "Cláusulas" dessa aula.)

**Definição — Lema dual (tautologia de uma FNC)**

Por **dualidade** com o lema da FND: uma disjunção de literais $l_1 \vee ... \vee l_n$ é **tautologia sse contém um par complementar** (algum $l_i$ é $\neg l_j$: se tem $p \vee \neg p$ lá dentro, é sempre verdadeira; se não tem, pondo cada literal a $F$ ela falha).
Logo, uma fórmula em FNC é **tautologia sse todas as suas cláusulas são tautologias** (uma conjunção é sempre verdadeira sse cada termo é sempre verdadeiro).

**Frequente em exame**

Guarda a **simetria**: FND serve para decidir **satisfazibilidade** (basta **uma** conjunção sem par complementar); FNC serve para decidir se é **tautologia** (**todas** as cláusulas têm de ter um par complementar). O contrário **não** é fácil: decidir se uma FNC é satisfazível é exatamente o problema SAT difícil.

Duas formas de obter uma FNC — as mesmas duas da FND, "ao contrário":

1. **A partir da tabela de verdade**, pelo método dual: escolher as linhas em que $f$ dá **F**; para cada uma, formar a **disjunção** em que a variável entra **negada se está a** $V$ nessa linha e **positiva se está a** $F$; tomar a **conjunção** dessas disjunções.
2. **A partir da fórmula**: passos 1 e 2 iguais aos da FND, mas no passo 3 usa-se a distributividade do $\vee$ **sobre o** $\wedge$:

$$(\phi \wedge \psi) \vee \theta \Leftrightarrow (\phi \vee \theta) \wedge (\psi \vee \theta) \qquad \theta \vee (\phi \wedge \psi) \Leftrightarrow (\theta \vee \phi) \wedge (\theta \vee \psi)$$

**Exemplo resolvido — FNC a partir de uma tabela (slides; a mesma função da FND acima)**

Agora interessam as linhas com **F** (4, 6, 7, 8). Para cada uma, a cláusula é construída para ser **falsa exatamente nessa linha**: cada literal tem de ser $F$ ali, por isso variável a $V$ → entra **negada**, variável a $F$ → entra **positiva**.

| linha | $x_1$ | $x_2$ | $x_3$ | $f$ | cláusula |
|---|---|---|---|---|---|
| 4 | V | F | F | F | $\neg p_1 \vee p_2 \vee p_3$ |
| 6 | F | V | F | F | $p_1 \vee \neg p_2 \vee p_3$ |
| 7 | F | F | V | F | $p_1 \vee p_2 \vee \neg p_3$ |
| 8 | F | F | F | F | $p_1 \vee p_2 \vee p_3$ |

Verificação da linha 4: com $x_1 = V, x_2 = F, x_3 = F$, $\neg p_1 \vee p_2 \vee p_3 = F \vee F \vee F = F$; e em qualquer outra linha pelo menos um literal é $V$. A conjunção das 4 cláusulas é $F$ exatamente nas linhas 4, 6, 7, 8 — ou seja, realiza $f$:

$$(\neg p_1 \vee p_2 \vee p_3) \wedge (p_1 \vee \neg p_2 \vee p_3) \wedge (p_1 \vee p_2 \vee \neg p_3) \wedge (p_1 \vee p_2 \vee p_3)$$

**Exemplo resolvido — FNC de $(p \vee r) \leftrightarrow (q \wedge \neg p)$ e é tautologia? (slides)**

Os passos 1 e 2 são **exatamente** os da FND (ver acima), e dão:

$$((\neg p \wedge \neg r) \vee (q \wedge \neg p)) \wedge ((\neg q \vee p) \vee (p \vee r))$$

O lado direito já é uma cláusula: $\neg q \vee p \vee p \vee r$ (associatividade). Falta o lado esquerdo, que é um $\vee$ de $\wedge$'s.
**Passo 3a** — distribuir com $\theta = \neg p \wedge \neg r$, $\phi = q$, $\psi = \neg p$, usando $\theta \vee (\phi \wedge \psi) \Leftrightarrow (\theta \vee \phi) \wedge (\theta \vee \psi)$:

$$(\neg p \wedge \neg r) \vee (q \wedge \neg p) \Leftrightarrow \big((\neg p \wedge \neg r) \vee q\big) \wedge \big((\neg p \wedge \neg r) \vee \neg p\big)$$

**Passo 3b** — cada bloco ainda tem um $\wedge$ dentro de um $\vee$; distribui-se outra vez $((a \wedge b) \vee c \Leftrightarrow (a \vee c) \wedge (b \vee c))$:

- $(\neg p \wedge \neg r) \vee q \Leftrightarrow (\neg p \vee q) \wedge (\neg r \vee q)$
- $(\neg p \wedge \neg r) \vee \neg p \Leftrightarrow (\neg p \vee \neg p) \wedge (\neg r \vee \neg p)$

**FNC final** (5 cláusulas):

$$(\neg p \vee q) \wedge (\neg r \vee q) \wedge (\neg p \vee \neg p) \wedge (\neg r \vee \neg p) \wedge (\neg q \vee p \vee p \vee r)$$

**É tautologia?** Verifica-se cada cláusula: $\neg p \vee q$ não tem par complementar $\rightarrow$ já não é tautologia, e a fórmula também **não** é (resposta dos slides: **Não**). Contraexemplo tirado da própria cláusula (pôr os seus literais a $F$): $p = V$, $q = F$ — de facto, com $p = V$ o lado $p \vee r$ é $V$ e $q \wedge \neg p$ é $F$, logo a equivalência é $F$. ✓

**Atenção**

Para "limpar" uma FND/FNC usam-se as leis extra da Aula 2 (complementaridade, elemento neutro/absorvente, absorção). Na FNC acima, por exemplo: $\neg p \vee \neg p \Leftrightarrow \neg p$ (idempotência), e a cláusula $\neg r \vee \neg p$ é absorvida por $\neg p$. Fica $(\neg p \vee q) \wedge (\neg r \vee q) \wedge \neg p \wedge (\neg q \vee p \vee r)$ — com $p = F$ forçado, sobra $(\neg r \vee q) \wedge (\neg q \vee r)$, ou seja $q \leftrightarrow r$: coerente com a verificação feita na FND. Uma FND/FNC não é única; qualquer uma equivalente está certa.

**Aplicação aos exercícios da prática: FND/FNC**

**Exemplo resolvido — Exercício 1.17(a) (lab, proplogic.pdf) — FND e FNC a partir da tabela**

A função (a) tem esta coluna (na ordem de linhas do enunciado, $p, q, r$ de $VVV$ até $FFF$): $V, V, V, V, F, V, V, V$. Só **uma** linha dá $F$: a linha $p = F, q = V, r = V$.
**FNC** (uma cláusula por linha com $F$ — só há uma): na linha $p = F, q = V, r = V$, $p$ está a $F \to$ entra positivo; $q$ e $r$ estão a $V \to$ entram negados:

$$p \vee \neg q \vee \neg r$$

**FND** (uma conjunção por linha com $V$ — há sete):

| $p$ | $q$ | $r$ | conjunção |
|---|---|---|---|
| V | V | V | $p \wedge q \wedge r$ |
| V | V | F | $p \wedge q \wedge \neg r$ |
| V | F | V | $p \wedge \neg q \wedge r$ |
| V | F | F | $p \wedge \neg q \wedge \neg r$ |
| F | V | F | $\neg p \wedge q \wedge \neg r$ |
| F | F | V | $\neg p \wedge \neg q \wedge r$ |
| F | F | F | $\neg p \wedge \neg q \wedge \neg r$ |

$$(p \wedge q \wedge r) \vee (p \wedge q \wedge \neg r) \vee (p \wedge \neg q \wedge r) \vee (p \wedge \neg q \wedge \neg r) \vee (\neg p \wedge q \wedge \neg r) \vee (\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge \neg q \wedge \neg r)$$

**Lição:** quando a tabela tem **poucos** $F$, a FNC sai muito mais curta; quando tem **poucos** $V$, é a FND. Ambas estão certas — as duas representam a mesma função (confirma: a FNC só falha em $p = F, q = V, r = V$, que é a única linha que falta na FND). Também se podia simplificar a FND de 7 termos, mas não é pedido.

**Exemplo resolvido — Exercício 1.18(d) (lab, proplogic.pdf) — FND e FNC de $(p \to q) \to (\neg p \to \neg q)$**

$$\begin{aligned}
& (p \to q) \to (\neg p \to \neg q) && \\
\Leftrightarrow\ & \neg(p \to q) \vee (\neg p \to \neg q) && \text{(implicação, na } \to \text{ exterior)} \\
\Leftrightarrow\ & \neg(\neg p \vee q) \vee (\neg\neg p \vee \neg q) && \text{(implicação, nas duas interiores)} \\
\Leftrightarrow\ & (\neg\neg p \wedge \neg q) \vee (\neg\neg p \vee \neg q) && \text{(De Morgan)} \\
\Leftrightarrow\ & (p \wedge \neg q) \vee p \vee \neg q && \text{(dupla negação, 2 vezes; associatividade)}
\end{aligned}$$

**FND:** $(p \wedge \neg q) \vee p \vee \neg q$ já é FND — três conjunções: $p \wedge \neg q$, $p$ (conjunção de um só literal), $\neg q$. Simplificando por absorção ($p \vee (p \wedge \neg q) \Leftrightarrow p$):

$$p \vee \neg q$$

**FNC:** distribui-se o $\vee$ sobre o $\wedge$ em $(p \wedge \neg q) \vee (p \vee \neg q)$, com $\theta = p \vee \neg q$:

$$(p \vee p \vee \neg q) \wedge (\neg q \vee p \vee \neg q) \Leftrightarrow (p \vee \neg q) \wedge (p \vee \neg q) \Leftrightarrow p \vee \neg q$$

(idempotência duas vezes). Repara que $p \vee \neg q$ é **ao mesmo tempo** FND (duas conjunções de um literal) e FNC (uma cláusula) — é normal com fórmulas pequenas.
Não é tautologia (a cláusula não tem par complementar): com $p = F, q = V$, $p \to q = V$ e $\neg p \to \neg q = V \to F = F$. (A alínea (k) do enunciado é **igual** a esta.)

**Exemplo resolvido — Exercício 1.18(h) (lab, proplogic.pdf) — FND e FNC de $(p \to q) \land (\neg q \to (p \lor \neg q))$**

**Eliminar implicações:** $p \to q \Leftrightarrow \neg p \lor q$; e $\neg q \to (p \lor \neg q) \Leftrightarrow \neg\neg q \lor p \lor \neg q \Leftrightarrow q \lor p \lor \neg q$.

$$(\neg p \lor q) \land (q \lor p \lor \neg q)$$

**FNC:** isto **já é** uma FNC (conjunção de duas cláusulas). Repara que a segunda cláusula contém $q$ e $\neg q$: é tautologia, logo vale sempre $V$ e pode sair ($\phi \land V \Leftrightarrow \phi$). FNC simplificada: $\neg p \lor q$.
**FND** (fazendo a distribuição toda, sem simplificar antes, para treinar): $(\neg p \lor q) \land X$ com $X = q \lor p \lor \neg q$ distribui como $(\neg p \land X) \lor (q \land X)$, e cada um distribui outra vez sobre os 3 literais de $X$:

$$(\neg p \land q) \lor (\neg p \land p) \lor (\neg p \land \neg q) \lor (q \land q) \lor (q \land p) \lor (q \land \neg q)$$

Apagam-se as conjunções com par complementar ($\neg p \land p$ e $q \land \neg q$) e $q \land q \Leftrightarrow q$:

$$(\neg p \land q) \lor (\neg p \land \neg q) \lor q \lor (p \land q)$$

Isto é uma FND correta. Simplificando por absorção ($q$ absorve $\neg p \land q$ e $p \land q$), sobra $(\neg p \land \neg q) \lor q$; e como $(\neg p \land \neg q) \lor q \Leftrightarrow (\neg p \lor q) \land (\neg q \lor q) \Leftrightarrow \neg p \lor q$, chega-se ao mesmo $\neg p \lor q$ da FNC — que também é FND. ✓
**Lição:** detetar cedo uma cláusula-tautologia (ou uma conjunção contraditória) poupa imenso trabalho de distribuição.

**Pratica agora — Formas normais (proplogic.pdf)**

- **1.17 (b)** e **(c)** — FND e FNC pela tabela. Na (c), olha bem para a coluna antes de começar: há uma fórmula muito mais curta escondida (as formas canónicas saem longas, mas simplificam-se).
- **1.18 (e)** — é a Lei de Peirce (Aula 2): o que acontece à FNC de uma tautologia?
- **1.18 (l)** — a mais comprida, boa para treinar a distributividade com 4 variáveis.
- **1.20** — o programa: valoração, tabela, classificação e FNC/FND (junta tudo o que viste até aqui).

As restantes alíneas de 1.18 são a mesma técnica; não precisas de as fazer todas (e repara que (a) = (c) e (d) = (k) no próprio enunciado).
*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

## Fórmulas de Horn

**Definição — Fórmula de Horn**

Uma **fórmula de Horn** é uma fórmula em **FNC** em que **cada cláusula tem no máximo um literal positivo**.
Exemplos (dos slides), com o nº de literais positivos por cláusula:

- $p \land \neg q \land (q \lor \neg p)$: cláusulas $p$ (1), $\neg q$ (0), $q \lor \neg p$ (1) → Horn.
- $(\neg p \lor \neg q \lor \neg s \lor p) \land (\neg q \lor \neg r \lor p) \land (\neg p \lor \neg s \lor s)$: 1, 1, 1 → Horn.
- $(\neg p \lor \neg q \lor \neg s) \land (\neg q \lor \neg r \lor p) \land s$: 0, 1, 1 → Horn.

Contra-exemplo: $(p \lor q) \land \neg r$ **não** é Horn — a cláusula $p \lor q$ tem dois literais positivos.

Cada cláusula de Horn pode ler-se como uma **implicação** (usando $\neg a \lor b \Leftrightarrow a \to b$ e De Morgan $\neg p_1 \lor ... \lor \neg p_n \Leftrightarrow \neg(p_1 \land ... \land p_n)$):

| forma da cláusula | lida como implicação | nome informal |
|---|---|---|
| $\neg p_1 \lor ... \lor \neg p_n \lor p$ | $(p_1 \land ... \land p_n) \to p$ | regra |
| $\neg p_1 \lor ... \lor \neg p_n$ (sem positivo) | $(p_1 \land ... \land p_n) \to F$ | restrição ("não podem ser todos $V$") |

| forma da cláusula | lida como implicação | nome informal |
|---|---|---|
| $p$ (sem negativos) | $V \to p$ | facto ($p$ tem de ser $V$) |

**Atenção**

**Nem todas as fórmulas têm uma fórmula de Horn equivalente** — basta que a sua FNC tenha uma cláusula com mais de um literal positivo que não possa ser simplificada (ex.: $p \vee q$). A vantagem das fórmulas de Horn é que a sua satisfazibilidade decide-se com um **algoritmo eficiente** (linear/polinomial), sem tabela de verdade.

### Algoritmo de satisfazibilidade para fórmulas de Horn

A ideia: começar com todas as variáveis "desconhecidas" e só atribuir $V$ a uma variável quando **for obrigatório** (quando, sem isso, uma cláusula ficaria falsa). Se essas atribuições obrigatórias fizerem uma cláusula ficar $F$, a fórmula é insatisfazível; senão, põem-se todas as restantes variáveis a $F$ e está encontrada uma valoração que a satisfaz.

**Definição — Algoritmo (versão dos slides)**

1. Escrever numa linha as variáveis e a fórmula (como numa linha de tabela de verdade, ainda vazia).
2. Se alguma **variável sozinha** é um dos elementos da conjunção (um **facto**, $V \to p$), atribuir-lhe $V$. **Porquê?** Porque uma conjunção só é $V$ se cada elemento for $V$; se $p$ é um elemento, tem de ser $p = V$ em qualquer valoração que satisfaça a fórmula.
3. Com os valores já conhecidos, avaliar cada cláusula. Se numa cláusula todos os literais negativos já estão a $F$ (as suas variáveis estão a $V$), então, para a cláusula ser $V$, o **literal positivo** tem de ser $V \to$ atribuir $V$ a essa variável.
4. Repetir o passo 3 até nada mais poder ser acrescentado.
5. Se algum elemento da conjunção ficou com valor $F$, a fórmula é $F \to$ **não satisfazível**. Caso contrário, é **satisfazível**: atribui-se $F$ a todas as variáveis restantes e a fórmula fica $V$.

**Exemplo resolvido — $p \wedge \neg q \wedge (q \vee \neg p)$ (slides)**

Cláusulas: $C_1 = p$, $C_2 = \neg q$, $C_3 = q \vee \neg p$ (lida como $p \to q$).
**Passo 1** — linha vazia:

| $p$ | $q$ | $C_1 = p$ | $C_2 = \neg q$ | $C_3 = q \vee \neg p$ |
|---|---|---|---|---|

**Passo 2** — $p$ é um elemento da conjunção (facto) $\to p = V$:

| $p$ | $q$ | $C_1 = p$ | $C_2 = \neg q$ | $C_3 = q \vee \neg p$ |
|---|---|---|---|---|
| V | | V | | $(\neg p = F)$ |

**Passo 3** — em $C_3$, o literal negativo $\neg p$ já é $F$; para $C_3$ ser $V$, o positivo $q$ tem de ser $V \to q = V$:

| $p$ | $q$ | $C_1 = p$ | $C_2 = \neg q$ | $C_3 = q \vee \neg p$ |
|---|---|---|---|---|
| V | V | V | | V |

**Passo 4** — repete-se: agora $C_2 = \neg q$ com $q = V$ dá $F$:

| $p$ | $q$ | $C_1 = p$ | $C_2 = \neg q$ | $C_3 = q \vee \neg p$ |
|---|---|---|---|---|
| V | V | V | **F** | V |

**Passo 5** — um elemento da conjunção ($C_2$) ficou $F \to$ a fórmula é $F \to$ **não é satisfazível**. (Não há escolha possível: $p = V$ e $q = V$ foram ambos **obrigatórios**.)

**Atenção**

Nota adicional — **porque é que o algoritmo está correto** (é o Exercício 1.21(b) do lab; nos slides fica só o "porquê?"):

- **Se diz "insatisfazível", está certo:** cada $V$ atribuído foi *obrigatório* — qualquer valoração que satisfaça a fórmula tem de o ter. Se as atribuições obrigatórias já tornam uma cláusula $F$, nenhuma valoração satisfaz a fórmula.
- **Se diz "satisfazível", está certo:** com as marcadas a $V$ e as outras a $F$, pega-se numa cláusula qualquer. (i) Se o seu literal positivo está marcado, ela é $V$. (ii) Senão, algum dos seus literais negativos $\neg x$ tem $x$ **não** marcado (se todos estivessem marcados, o passo 3 teria obrigado a marcar o positivo, ou — se não houver positivo — a cláusula seria $F$ e o algoritmo teria parado no passo 5); logo $x = F$ e $\neg x = V$, e a cláusula é $V$.

É exatamente aqui que se usa "no máximo um literal positivo": pôr as restantes a $F$ torna verdadeiros **todos** os literais negativos por decidir, e nunca é preciso "escolher" entre dois positivos. Por isso nunca há retrocesso (*backtracking*) e o algoritmo é eficiente.

**Exemplo resolvido — O mesmo algoritmo em pseudo-código (base para o Exercício 1.21(a))**

Vendo cada cláusula como implicação $(p_1 \wedge \ldots \wedge p_n) \to h$, com $h$ uma variável ou $F$ (e $n = 0$ para factos):

```
marcadas := {}                          -- variáveis que TÊM de ser V
repetir
  mudou := falso
  para cada cláusula (p1 e ... e pn) -> h:
    se p1,...,pn estão todos em marcadas:   -- antecedente todo V
      se h = F: devolver INSATISFAZÍVEL
      se h não está em marcadas:
        acrescentar h a marcadas; mudou := verdadeiro
até não mudou
devolver SATISFAZÍVEL   -- v(x)=V se x em marcadas, senão v(x)=F
```

Cada iteração do `repetir` marca pelo menos uma variável nova (ou pára), por isso há no máximo (nº de variáveis + 1) iterações — polinomial.

**Exemplo resolvido — Exercício 1.21(c), 1.ª e 2.ª fórmulas (lab, proplogic.pdf)**

**(i)** $(\neg p \vee \neg q) \wedge (\neg q \vee r) \wedge q$**.** Como implicações: $(p \wedge q) \to F$, $q \to r$, $V \to q$.

- Facto $q \to q = V$.
- $q \to r$: antecedente $q$ marcado $\to r = V$.
- $(p \wedge q) \to F$: antecedente precisa de $p$ **e** $q$; $p$ não está marcado → não dispara.
- Nova volta: nada muda. Nenhuma cláusula ficou $F$.

**Satisfazível**, com $q = V$, $r = V$ e a restante $p = F$. Verificação: $\neg p \vee \neg q = V \vee F = V$; $\neg q \vee r = F \vee V = V$; $q = V$. ✓

**(ii)** $p \wedge (\neg p \vee q) \wedge (\neg q \vee p)$**.** Como implicações: $V \to p$, $p \to q$, $q \to p$.

- Facto $p \to p = V$.
- $p \to q$: $p$ marcado $\to q = V$.
- $q \to p$: $q$ marcado, mas $p$ já estava marcado → nada novo.
- Nova volta: nada muda; nenhuma cláusula ficou $F$.

**Satisfazível**, com $p = q = V$ (não sobram variáveis para pôr a $F$). Verificação: $p = V$; $\neg p \vee q = F \vee V = V$; $\neg q \vee p = F \vee V = V$. ✓

**Pratica agora — Horn (proplogic.pdf)**

- **1.21 (c)**, 3.ª e 4.ª fórmulas: $\neg p \wedge (\neg p \vee q) \wedge \neg q$ (o que acontece quando **não há nenhum facto**?) e $p \wedge (\neg p \vee q) \wedge \neg r$.
- **1.21 (a)** — implementar o algoritmo a partir do pseudo-código acima.

*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

Os slides da Aula 4 começam por rever as fórmulas de Horn e o algoritmo acima (é a mesma matéria, por isso está toda aqui). O exercício dos slides dessa aula, $p \wedge (\neg p \vee q) \wedge \neg r$, é a 4.ª fórmula da 1.21(c), na caixa acima.

# Aula 4 — Satisfazibilidade: cláusulas e o algoritmo de Davis-Putnam

Os slides desta aula começam por rever as formas normais (negativa, disjuntiva, conjuntiva), a satisfazibilidade de uma FND e as fórmulas de Horn com o seu algoritmo. Isso já está explicado na Aula 3 ("Fórmulas de Horn"). A matéria nova é o caso **geral**: decidir se uma fórmula qualquer em FNC é satisfazível, com o **algoritmo de Davis-Putnam (DLL)**.

## O problema SAT

Saber se uma fórmula da lógica proposicional é satisfazível é um problema **central na informática**, porque muitos problemas de outras áreas se podem escrever como uma fórmula e a pergunta "há solução?" passa a ser "a fórmula é satisfazível?":

- **otimização**: planeamento, escalonamento de horários (*scheduling*), etc.;
- **combinatória**: coloração de grafos, etc.;
- **teorias da lógica de primeira ordem**: programação linear, aritmética de números reais, sequências de bits, apontadores, etc. (os ***SMT solvers***).

Se a fórmula não está em FND, nem numa classe em que a satisfazibilidade seja fácil (como Horn), a tabela de verdade é o único caminho? **No pior caso, sim**: não se conhece nenhum algoritmo para a satisfazibilidade que não seja **exponencial** no número de variáveis e conectivos (**SAT é NP-completo**; ver também a nota da Aula 3 sobre "P = NP?").

Mas, **no caso geral**, há algoritmos muito melhores do que construir a tabela inteira. A ideia é **ir construindo uma valoração parcial** que satisfaça a fórmula, em vez de gerar valorações completas (linhas da tabela) e testar cada uma. Isto fica especialmente simples com fórmulas em **FNC**, escritas de forma compacta como conjuntos de **cláusulas**.

## Cláusulas

**Definição — Cláusula**

Uma **cláusula** é uma disjunção de literais $l_1 \vee l_2 \vee \ldots \vee l_n$, com $n \geq 0$ (é o mesmo nome da Aula 3, "Forma normal conjuntiva"). Representa-se pelo **conjunto** dos seus literais $\{l_1, \ldots, l_n\}$.

- Se $n = 0$, a cláusula é **vazia** e corresponde a $F$.
- Se $n = 1$, a cláusula é **unitária**.

Exemplo: $p \vee \neg q \vee \neg p \vee s$ representa-se por $\{p, \neg q, \neg p, s\}$.
Qualquer fórmula em **FNC** representa-se por um **conjunto de cláusulas** (a conjunção fica implícita).

**Exemplo resolvido**

$$\neg p \wedge (q \vee r \vee q) \wedge (\neg r \vee \neg s) \wedge (p \vee s) \wedge (\neg q \vee \neg s)$$

corresponde ao conjunto de cláusulas

$$\{\{\neg p\},\ \{q, r\},\ \{\neg r, \neg s\},\ \{p, s\},\ \{\neg q, \neg s\}\}$$

Repara na segunda cláusula: $q \vee r \vee q$ fica $\{q, r\}$, porque um conjunto não tem repetidos (é a idempotência, $q \vee q \Leftrightarrow q$). Pela mesma razão, uma cláusula repetida no conjunto conta uma só vez (idempotência do $\wedge$).

**Atenção — O que quer dizer um conjunto vazio**

Nota adicional (não estava explícito nos slides, mas o algoritmo DLL abaixo depende disto): há dois "vazios" diferentes.

- **Cláusula vazia** $\{\}$ (escreve-se $F$): uma disjunção sem nenhum literal é **falsa**. Se um conjunto de cláusulas contém a cláusula vazia, é **insatisfazível** (uma conjunção com um $F$ é $F$).
- **Conjunto de cláusulas vazio** $\{\}$: uma conjunção sem nenhum termo é **verdadeira** (não há nenhuma cláusula para falhar). É **satisfazível**.

**Definição — Literal complementar**

Dado um literal $l$, o **literal complementar** $\tilde{l}$ é:

$$\tilde{l} = \begin{cases} \neg l, & \text{se } l \text{ é uma variável (positivo)} \\ p, & \text{se } l \text{ é da forma } \neg p \text{ (negativo)} \end{cases}$$

Por exemplo, $\tilde{p} = \neg p$ e $\widetilde{\neg p} = p$.

Como se viu na Aula 3 (lema dual), uma cláusula é **tautologia** se contém um par de literais complementares $p$ e $\neg p$. Essas cláusulas **podem ser retiradas** do conjunto sem alterar a satisfazibilidade: são sempre $V$, e $V \wedge \varphi \Leftrightarrow \varphi$. Por exemplo, $\{p, \neg q, \neg p, s\}$ acima pode simplesmente sair.

## Propagação unitária

O algoritmo de **Davis-Putnam** tem uma versão inicial de **1960** e ainda é a base de muitos dos algoritmos mais eficientes de hoje (ver as competições de *SAT solvers*, http://www.satcompetition.org/). A ideia é **considerar os valores possíveis para cada variável e simplificar a fórmula** com essas atribuições, até se poder concluir se é satisfazível. A simplificação principal é a **propagação unitária**.

**Definição — Propagação unitária**

Seja $S$ um conjunto de cláusulas. Obtém-se $S'$ a partir de $S$ por **propagação unitária** repetindo a seguinte transformação: se $S$ contém uma **cláusula unitária** $l$, então

1. **retiram-se** de $S$ todas as cláusulas da forma $l \vee C'$ (as que contêm $l$, incluindo a própria cláusula $l$);
2. **substitui-se** cada cláusula da forma $\tilde{l} \vee C'$ pela cláusula $C'$ (tira-se o literal complementar).

**Atenção — Porque é que a propagação não muda a satisfazibilidade**

Nota adicional (não estava explícito nos slides). Se $l$ é uma cláusula sozinha, qualquer valoração que satisfaça $S$ tem de pôr $l = V$ (é um elemento da conjunção, como os factos de Horn). Com $l = V$:

- uma cláusula que contém $l$ fica $V$ de certeza: pode sair;
- numa cláusula que contém $\tilde{l}$, esse literal é $F$ e não ajuda nada ($F \vee C' \Leftrightarrow C'$): pode sair da cláusula.

Se ao tirar $\tilde{l}$ uma cláusula fica **vazia** (era só $\tilde{l}$), aparece $F$: o conjunto é insatisfazível. Os literais propagados dizem também **que valores** têm as variáveis numa valoração que satisfaça $S$.

**Exemplo resolvido — Exemplo dos slides: só com propagação (1/2)**

$$S = \{p_1,\ \neg p_1 \vee \neg p_2,\ p_3 \vee p_2,\ \neg p_7 \vee p_2,\ \neg p_3 \vee p_4,\ \neg p_3 \vee p_5,$$
$$\neg p_4 \vee \neg p \vee q,\ \neg p_5 \vee \neg p_6 \vee r,\ \neg p \vee \neg q \vee p_6,\ p \vee p_7,\ \neg r \vee p_7\}$$

**Propagar $p_1$.** Saem as cláusulas com $p_1$: só a própria $p_1$. Às que têm $\neg p_1$ tira-se esse literal: $\neg p_1 \vee \neg p_2$ fica $\neg p_2$.

$$\{\neg p_2,\ p_3 \vee p_2,\ \neg p_7 \vee p_2,\ \neg p_3 \vee p_4,\ \neg p_3 \vee p_5,\ \neg p_4 \vee \neg p \vee q,\ \neg p_5 \vee \neg p_6 \vee r,\ \neg p \vee \neg q \vee p_6,\ p \vee p_7,\ \neg r \vee p_7\}$$

**Propagar $\neg p_2$.** Sai $\neg p_2$. O complementar é $p_2$: $p_3 \vee p_2$ fica $p_3$ e $\neg p_7 \vee p_2$ fica $\neg p_7$.

$$\{p_3,\ \neg p_7,\ \neg p_3 \vee p_4,\ \neg p_3 \vee p_5,\ \neg p_4 \vee \neg p \vee q,\ \neg p_5 \vee \neg p_6 \vee r,\ \neg p \vee \neg q \vee p_6,\ p \vee p_7,\ \neg r \vee p_7\}$$

**Propagar $p_3$.** Sai $p_3$. $\neg p_3 \vee p_4$ fica $p_4$; $\neg p_3 \vee p_5$ fica $p_5$.
**Propagar $\neg p_7$.** Sai $\neg p_7$. $p \vee p_7$ fica $p$; $\neg r \vee p_7$ fica $\neg r$.

$$\{p_4,\ p_5,\ \neg p_4 \vee \neg p \vee q,\ \neg p_5 \vee \neg p_6 \vee r,\ \neg p \vee \neg q \vee p_6,\ p,\ \neg r\}$$

**Exemplo resolvido — Exemplo dos slides: só com propagação (2/2)**

**Propagar $p_4$.** $\neg p_4 \vee \neg p \vee q$ fica $\neg p \vee q$.
**Propagar $p_5$.** $\neg p_5 \vee \neg p_6 \vee r$ fica $\neg p_6 \vee r$.
**Propagar $\neg r$.** $\neg p_6 \vee r$ fica $\neg p_6$.
**Propagar $p$.** Sai $p$. $\neg p \vee q$ fica $q$; $\neg p \vee \neg q \vee p_6$ fica $\neg q \vee p_6$.

$$\{q,\ \neg p_6,\ \neg q \vee p_6\}$$

**Propagar $q$.** $\neg q \vee p_6$ fica $p_6$: $\{\neg p_6,\ p_6\}$.
**Propagar $\neg p_6$.** Sai $\neg p_6$. A cláusula $p_6$ perde o seu único literal e fica **vazia**:

$$\{F\}$$

O conjunto inicial é **insatisfazível**, e aqui bastou a propagação unitária (nunca foi preciso "adivinhar" um valor). Os slides propagam alguns literais ao mesmo tempo ($p_3$ e $\neg p_7$; depois $p_4$, $p_5$, $\neg r$ e $p$); a ordem não altera o resultado.

## O algoritmo DLL

Quando a propagação pára e ainda não se chegou a uma conclusão (não há cláusulas unitárias, o conjunto não é vazio e não contém $F$), é preciso **escolher** um literal e experimentar os dois valores.

**Definição — Algoritmo de Davis-Putnam (DLL)**

```
DLL(S)
  input:  conjunto de clausulas S
  output: satisfazivel ou insatisfazivel
  S := propagate(S)
  if S vazio then return satisfazivel
  if S contem F then return insatisfazivel
  l := select_literal(S)
  if DLL(S uniao {l}) = satisfazivel
    then return satisfazivel
    else return DLL(S uniao {~l})       -- ~l: o complementar de l
```

- Acrescentar a cláusula unitária $\{l\}$ é **supor** $l = V$; a propagação da chamada recursiva trata do resto.
- Se essa suposição dá insatisfazível, **volta-se atrás** (*backtracking*) e supõe-se o contrário, $\tilde{l}$.
- Só se responde "insatisfazível" quando **os dois ramos** falham.

A função `select_literal` devolve um literal de uma das cláusulas. Pode ver-se como um **parâmetro** do algoritmo: uma boa escolha torna-o muito mais eficiente. Critérios possíveis, escolher uma variável que:

- **ocorre mais vezes**;
- tem o **produto** das ocorrências de $l$ e de $\tilde{l}$ **máximo**;
- ocorre mais vezes em **cláusulas de tamanho mínimo**;
- etc.

**Atenção — Ler uma valoração da resposta**

Nota adicional (não estava explícito nos slides). Quando o DLL responde "satisfazível", os literais que foram **supostos** e **propagados** no ramo que teve sucesso formam uma valoração que satisfaz $S$. As variáveis que não apareceram podem ter qualquer valor.

**Exemplo resolvido — Exemplo dos slides: é preciso escolher**

$$S = \{\neg p \vee \neg q,\ \neg p \vee q,\ p \vee \neg q,\ p \vee q\}$$

Não há cláusulas unitárias: a propagação não faz nada. Escolhe-se um literal, por exemplo $\neg p$.
**Ramo $S \cup \{\neg p\}$.** Propagar $\neg p$: saem as cláusulas com $\neg p$ ($\neg p \vee \neg q$ e $\neg p \vee q$). Às que têm $p$ tira-se $p$: $p \vee \neg q$ fica $\neg q$ e $p \vee q$ fica $q$. Obtém-se $\{\neg q, q\}$. Propagar $\neg q$: sai $\neg q$ e $q$ fica vazia: $\{F\}$. **Falha.**
**Ramo $S \cup \{p\}$.** Propagar $p$: saem $p \vee \neg q$ e $p \vee q$; $\neg p \vee \neg q$ fica $\neg q$ e $\neg p \vee q$ fica $q$. Outra vez $\{\neg q, q\} \Rightarrow \{F\}$. **Falha.**
Os dois ramos falham: o algoritmo devolve **insatisfazível**. (Faz sentido: as quatro cláusulas proíbem cada uma das quatro combinações de valores de $p$ e $q$.)

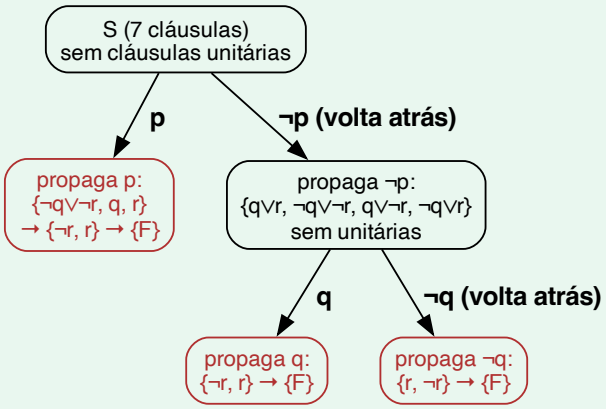
**Exemplo resolvido — Exercício 1.22(a) (lab, proplogic.pdf)**

$$S = \{p \vee q \vee r,\ \neg p \vee \neg q \vee \neg r,\ p \vee \neg q \vee \neg r,\ p \vee q \vee \neg r,\ \neg p \vee q,\ \neg p \vee r,\ p \vee \neg q \vee r\}$$

Não há unitárias. Critério "ocorre mais vezes": $p$ está nas 7 cláusulas, $q$ e $r$ em 6. Escolhe-se $p$.
**Ramo $p$.** Saem as 4 cláusulas com $p$. Às 3 com $\neg p$ tira-se $\neg p$: $\{\neg q \vee \neg r,\ q,\ r\}$. Propagar $q$: $\neg q \vee \neg r$ fica $\neg r$, $\{\neg r, r\}$. Propagar $\neg r$: $r$ fica vazia, $\{F\}$. **Falha.**
**Ramo $\neg p$.** Saem as 3 cláusulas com $\neg p$. Às 4 com $p$ tira-se $p$: $\{q \vee r,\ \neg q \vee \neg r,\ q \vee \neg r,\ \neg q \vee r\}$. Não há unitárias; $q$ e $r$ ocorrem 4 vezes cada. Escolhe-se $q$.

- **Ramo $q$**: saem $q \vee r$ e $q \vee \neg r$; $\neg q \vee \neg r$ fica $\neg r$ e $\neg q \vee r$ fica $r$. $\{\neg r, r\} \Rightarrow \{F\}$. **Falha.**
- **Ramo $\neg q$**: saem $\neg q \vee \neg r$ e $\neg q \vee r$; $q \vee r$ fica $r$ e $q \vee \neg r$ fica $\neg r$. $\{r, \neg r\} \Rightarrow \{F\}$. **Falha.**

Todos os ramos falham: **insatisfazível**.




**Pratica agora — Davis-Putnam (proplogic.pdf, 1.22 e 1.23)**

- **1.22 (c)** $\{p \vee \neg q \vee \neg r,\ \neg p \vee q \vee r,\ \neg p \vee q \vee r,\ \neg p \vee q,\ \neg p \vee r\}$. Repara na cláusula repetida. Depois de escolher o primeiro literal, a propagação faz o resto.
- **1.22 (e)** $\{p \vee \neg q \vee \neg r,\ \neg p \vee q \vee r,\ \neg p \vee \neg q \vee \neg r,\ \neg p \vee \neg q \vee r,\ \neg p \vee q \vee \neg r,\ p \vee q \vee \neg r\}$. Começa por $p$, como na 1.22(a); aqui é preciso voltar atrás antes de encontrar uma valoração. Diz qual é.
- **1.23** Implementar o algoritmo DLL (propagação + escolha + volta atrás), a partir do pseudo-código acima.

A 1.22(b) e a 1.22(d) são a mesma técnica com outros conjuntos; não é preciso fazê-las todas.
*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

# Aulas 5–6 — Dedução Natural

Os slides destas duas aulas vêm num único ficheiro ("Deduction"). Até aqui (Aulas 1–4) tratámos as duas primeiras partes de uma lógica: a **linguagem** (o que é uma fórmula) e a **semântica** (valorações, $\vDash_v$, $\vDash$). Agora passamos à terceira parte, o **sistema de dedução**: regras puramente sintáticas para obter fórmulas a partir de outras, sem falar em valorações. O sistema estudado é a **dedução natural (DN)**, escrito na **notação de Fitch**. É este o formato pedido em **todos** os exercícios da folha 2 ("All deductions must use the DN natural deduction system and the Fitch notation given in class").

## Semântica versus dedução

Há duas maneiras de dizer que uma conclusão $\phi$ "segue" de um conjunto de premissas $\Gamma$:

| | Semântica | Dedução |
|---|---|---|
| Notação | $\Gamma \vDash \phi$ | $\Gamma \vdash \phi$ |
| Lê-se | $\phi$ é consequência semântica de $\Gamma$ | $\phi$ deduz-se de $\Gamma$ |
| Significa | toda a valoração que satisfaz $\Gamma$ satisfaz $\phi$ | existe uma dedução (sequência de passos, cada um dado por uma regra) de $\phi$ a partir de $\Gamma$ |
| Natureza | fala de valores de verdade | só manipula símbolos (**sintática**) |

As duas setas que ligam as duas colunas têm nome:

$$\Gamma \vDash \phi \xrightarrow{\text{completude}} \Gamma \vdash \phi$$

$$\Gamma \vdash \phi \xrightarrow{\text{integridade } (\textit{soundness})} \Gamma \vDash \phi$$

**Definição — Integridade e completude**

Nota adicional (os slides só desenham as setas; as definições vêm dos apontamentos de Nelma Moreira, secção 1.5):

- Um sistema dedutivo é **íntegro** (ou *correto*, *sound*) se tudo o que se deduz é consequência semântica: se $\Gamma \vdash \phi$ então $\Gamma \vDash \phi$. Em particular, se $\vdash \phi$ então $\phi$ é tautologia. ("O que se deduz é verdade.")
- É **completo** se tudo o que é consequência semântica se deduz: se $\Gamma \vDash \phi$ então $\Gamma \vdash \phi$. Em particular, toda a tautologia tem uma dedução sem premissas. ("O que é verdade deduz-se.")

A dedução natural DN é **íntegra e completa** para a lógica proposicional. Logo, para qualquer $\Gamma$ e $\phi$: $\Gamma \vdash \phi$ sse $\Gamma \vDash \phi$.

**Porque é que se quer um sistema de dedução, se já temos a semântica?** Para ter um método automático, ou pelo menos semi-automático, de mostrar que uma fórmula é válida ou consequência de outras, sem percorrer todas as valorações (na lógica proposicional a tabela de verdade ainda é finita; na lógica de primeira ordem, mais à frente, deixa de ser). Uma dedução é um **método sintático**: cada passo verifica-se só a olhar para a forma das fórmulas.

**Frequente em exame**

**Como se mostra que** $\Gamma \nvdash \phi$ (não existe dedução)? Não basta "não conseguir encontrar uma". Usa-se a **integridade**: se existisse uma dedução, ter-se-ia $\Gamma \vDash \phi$. Por isso basta dar um **contraexemplo**, uma valoração $v$ com $\vDash_v \gamma$ para todo o $\gamma \in \Gamma$ e $\nvDash_v \phi$ (verificada com as regras de $\vDash_v$, como na Aula 2). Conclusão: $\Gamma \nvDash \phi$, logo, pela integridade de DN, $\Gamma \nvdash \phi$. É assim que se respondem as alíneas "não é válida" das 2.4, 2.9 e 2.10.

## Raciocínios válidos e contraexemplos

Informalmente, um **raciocínio** é uma sequência de afirmações em que uma delas, a **conclusão**, deve ser consequência das outras, as **premissas**. Os slides escrevem-no em "caixa", com as premissas por cima de uma linha horizontal e a conclusão por baixo:

| | |
|---|---|
| 1 | Se o Paulo adormecer e alguém o acordar, ele pragueja |
| 2 | O Paulo adormeceu |
| 3 | Ele não disse palavrões |
| 4 | **Logo:** ninguém o acordou |

Só interessa a **estrutura lógica** das frases. Com $p$ = "o Paulo adormece", $q$ = "alguém o acorda" e $r$ = "ele pragueja", o raciocínio é

$$(p \wedge q) \to r, \quad p, \quad \neg r \quad \text{logo} \quad \neg q.$$

**Definição**

- A conclusão é **consequência lógica** das premissas se é verdadeira **sempre que** as premissas são todas verdadeiras (é o $\{(p \wedge q) \to r,\ p,\ \neg r\} \vDash \neg q$ da Aula 2). Nesse caso o raciocínio é **válido**.
- Um raciocínio válido cujas premissas são **de facto** verdadeiras diz-se **sólido** (*sound*).
- Para mostrar que uma conclusão **não** é consequência das premissas, mostra-se uma situação em que as premissas são todas verdadeiras e a conclusão falsa: um **contraexemplo**.
- Para mostrar que **é** consequência, constrói-se uma **sucessão de passos** em que cada passo é consequência dos anteriores e das premissas. Formalizar isto dá os **sistemas de dedução**.

**Atenção**

"*Sound*" aparece com dois sentidos: um **raciocínio** sólido (válido e com premissas verdadeiras) e um **sistema dedutivo** íntegro (tudo o que deduz é consequência semântica, secção anterior). São coisas diferentes.

O segundo raciocínio dos slides: "Se chumbas, tens de repetir. Se não estudas, chumbas. Não estás a repetir. Logo: estudas, ou chumbas, ou as duas." Com $r$ = "chumbas", $t$ = "repetes" e $e$ = "estudas":

$$r \to t, \quad \neg e \to r, \quad \neg t \quad \text{logo} \quad e \vee r \vee (e \wedge r).$$

Também é **válido**. A dedução fica como exercício no fim da secção da implicação, depois de dadas as regras.

## Sistemas dedutivos axiomáticos

No início do séc. XX, **Hilbert** lançou as bases da teoria da prova moderna, com os **sistemas dedutivos axiomáticos**.

**Definição — Sistema dedutivo axiomático $\mathcal{D}$ e dedução**

Um sistema dedutivo $\mathcal{D}$ é formado por:
- **axiomas** (lógicos): fórmulas de base que traduzem as propriedades dos conectivos;
- **regras de inferência**: maneiras de obter fórmulas a partir de outras.

Uma **dedução** de $\phi_n$ em $\mathcal{D}$ a partir de um conjunto de fórmulas $\Sigma$ é uma sucessão **finita** de fórmulas $\phi_1, \dots, \phi_n$ em que cada $\phi_i$ $(1 \leq i \leq n)$:
- pertence a $\Sigma$ (é uma premissa), **ou**
- é um axioma, **ou**
- resulta de fórmulas anteriores $\phi_1 \dots \phi_{i-1}$ por uma regra de inferência.

Escreve-se $\Sigma \vdash_{\mathcal{D}} \phi_n$.
- **Teorema e prova.** Se $\Sigma = \emptyset$, $\phi_n$ é um **teorema** de $\mathcal{D}$, escreve-se $\vdash_{\mathcal{D}} \phi_n$, e a dedução $\phi_1, \dots, \phi_n$ chama-se uma **prova** de $\phi_n$.
- Se $\Sigma = \{\theta_1, \dots, \theta_n\}$ é finito, escreve-se $\theta_1, \dots, \theta_n \vdash_{\mathcal{D}} \phi$ e, quando o sistema é claro pelo contexto, só $\theta_1, \dots, \theta_n \vdash \phi$.

**Definição — Equivalência dedutiva ($\dashv\vdash$)**

Nota adicional (não está nos slides, mas a folha 2 usa-a nas 2.11 e 2.12; vem dos apontamentos de Nelma Moreira, 1.4.3.11): $\phi$ e $\psi$ são **dedutivamente equivalentes**, $\phi \dashv\vdash \psi$, se $\phi \vdash \psi$ **e** $\psi \vdash \phi$. Mostrar uma equivalência dedutiva são **duas deduções**, uma em cada sentido ("you have to show that the second formula is deduced from the first formula and vice versa").

## Dedução natural (DN) e a notação de Fitch

O sistema de **dedução natural**, DN, foi inventado por **G. Gentzen** (1935). As regras querem imitar a forma como se raciocina numa prova matemática. Em relação a um sistema à Hilbert tem duas diferenças:

- **não tem axiomas**, só regras de inferência;
- permite **introduzir hipóteses a meio da dedução** ("suponhamos que …"), que têm de ser **descartadas** antes de a dedução acabar.

Para cada conectivo há dois tipos de regras: de **introdução** (como se chega a uma fórmula com esse conectivo) e de **eliminação** (o que se pode tirar de uma fórmula com esse conectivo). Os conectivos tratados são $\wedge$, $\vee$, $\neg$, $\mathbf{F}$ e $\rightarrow$.

**Exemplo resolvido — A ideia, num exemplo informal (slide 16)**

Porque é que $(p \vee (q \wedge r)) \rightarrow ((p \vee q) \wedge (p \vee r))$ é uma tautologia? Raciocínio "natural":
- **Supõe-se** o antecedente, $p \vee (q \wedge r)$, e tenta-se chegar ao consequente.
- O antecedente é uma disjunção, por isso há **dois casos**:
  - **caso** $p$: então $p \vee q$ (basta um lado de uma disjunção), $p \vee r$, logo $(p \vee q) \wedge (p \vee r)$;
  - **caso** $q \wedge r$: então $q$ e $r$; daí $p \vee q$ e $p \vee r$, logo $(p \vee q) \wedge (p \vee r)$.
- Nos dois casos chega-se ao consequente. Como **supondo** o antecedente se obteve o consequente, conclui-se a implicação.

Cada "então" acima é uma regra de DN. A mesma prova, escrita formalmente, está no fim da secção da implicação.

**Representação das regras.** Uma regra de inferência "de $\phi_1 \dots \phi_k$ infere-se $\phi$" desenha-se como uma fração, com as premissas da regra em cima e a conclusão em baixo:

$$\frac{\phi_1 \quad \cdots \quad \phi_k}{\phi}$$

Como uma dedução é uma sequência de aplicações de regras, pode ser desenhada como uma **árvore**: as

folhas são as premissas e a raiz é a conclusão. Por exemplo, $p \wedge q,\ r \vdash q \wedge r$:

$$\dfrac{\dfrac{p \wedge q}{q}\,\wedge\mathrm{E}_2 \qquad r}{q \wedge r}\,\wedge\mathrm{I}$$

Estas árvores ficam muito grandes depressa. Por isso usa-se a **notação de Fitch**, uma representação **linear**:

**Definição — Notação de Fitch**

- Cada passo numa linha **numerada**.
- À direita, a **justificação**: a regra aplicada e os **números das linhas** usadas (ex.: "∧E, 1"; "∧I, 3, 2").
- As **premissas** vêm primeiro, sem justificação, e por baixo da última há um **traço horizontal**.
- A **barra vertical** à esquerda marca o alcance das hipóteses. Uma **sub-dedução** (hipótese nova) abre uma barra mais à direita, com a hipótese sublinhada no topo. Quando a sub-dedução acaba, a barra fecha.
- Uma regra que usa uma sub-dedução inteira cita-a como um intervalo de linhas (ex.: "∨E, 1, 2–3, 4–5").

O exemplo da árvore acima, em Fitch:

| | | |
|---|---|---|
| 1 | $p \wedge q$ | |
| 2 | $\underline{r}$ | |
| 3 | $q$ | ∧E, 1 |
| 4 | $q \wedge r$ | ∧I, 3, 2 |

A tabela com **todas** as regras de DN em Fitch está na última página da folha 2 (`proplogicproofs.pdf`, na pasta `Praticas/Semana_3/`). Nas secções seguintes cada regra é explicada uma a uma.

## Regras da conjunção

**Definição — Introdução e eliminação de ∧**

$$\dfrac{\phi \qquad \psi}{\phi \wedge \psi}\,\wedge\mathrm{I} \qquad \dfrac{\phi \wedge \psi}{\phi}\,\wedge\mathrm{E}_1 \qquad \dfrac{\phi \wedge \psi}{\psi}\,\wedge\mathrm{E}_2$$

- ∧**I**: se já se deduziu $\phi$ **e** já se deduziu $\psi$ (em quaisquer linhas anteriores, por qualquer ordem), pode deduzir-se $\phi \wedge \psi$.
- ∧**E**: de $\phi \wedge \psi$ pode deduzir-se $\phi$, e também $\psi$. Em Fitch as duas versões chamam-se só "∧E".

**Exemplo resolvido — $(p \wedge q) \wedge r \vdash r \wedge q$ (slide 22)**

A premissa é uma conjunção de uma conjunção. Desmonta-se com ∧E até se ter as peças $r$ e $q$ e monta-se o resultado com ∧I:

| | | |
|---|---|---|
| 1 | $\underline{(p \wedge q) \wedge r}$ | |
| 2 | $p \wedge q$ | ∧E, 1 |
| 3 | $r$ | ∧E, 1 |
| 4 | $q$ | ∧E, 2 |
| 5 | $r \wedge q$ | ∧I, 3, 4 |

Na linha 5, ∧I cita "3, 4" porque $r$ (linha 3) é o lado esquerdo e $q$ (linha 4) o direito.

**Pratica agora — Conjunção (proplogicproofs.pdf)**

- **2.11 (c)** $(\psi \wedge \phi) \wedge \delta \dashv\vdash \psi \wedge (\phi \wedge \delta)$. São duas deduções (uma em cada sentido), só com $\wedge$E e $\wedge$I.

*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

## Regras da disjunção

**Definição — Introdução e eliminação de $\vee$**

$$\frac{\phi}{\phi \vee \psi}\,\vee\mathrm{I}_1 \qquad \frac{\psi}{\phi \vee \psi}\,\vee\mathrm{I}_2 \qquad \frac{\phi \vee \psi \qquad \begin{array}{c}[\phi]\\ \vdots\\ \gamma\end{array} \qquad \begin{array}{c}[\psi]\\ \vdots\\ \gamma\end{array}}{\gamma}\,\vee\mathrm{E}$$

- $\vee$**I**: se já se deduziu $\phi$, pode deduzir-se **qualquer** disjunção que tenha $\phi$ como um dos lados ($\phi \vee \psi$ ou $\psi \vee \phi$, com $\psi$ qualquer).
- $\vee$**E** (raciocínio **por casos**): se
  1. já se deduziu $\phi \vee \psi$,
  2. **supondo** $\phi$ se deduz $\gamma$ (numa sub-dedução), e
  3. **supondo** $\psi$ se deduz $\gamma$ (noutra sub-dedução),

  então pode deduzir-se $\gamma$. A notação $[\phi]$ quer dizer "início de uma sub-dedução com a hipótese $\phi$", que só vale dentro dela.

Em Fitch, $\vee$E cita **três** coisas: a linha da disjunção e os dois intervalos das sub-deduções. As duas sub-deduções têm de acabar **na mesma fórmula** $\gamma$.

**Exemplo resolvido — $(p \wedge q) \vee (q \wedge r) \vdash q$ (slide 25)**

Premissa é uma disjunção: dois casos. Em cada um tira-se $q$ com $\wedge$E.

| | | |
|---|---|---|
| 1 | $(p \wedge q) \vee (q \wedge r)$ | |
| 2 | $\quad p \wedge q$ | |
| 3 | $\quad q$ | $\wedge$E, 2 |
| 4 | $\quad q \wedge r$ | |
| 5 | $\quad q$ | $\wedge$E, 4 |
| 6 | $q$ | $\vee$E, 1, 2–3, 4–5 |

- Linhas 2–3: caso 1, hipótese $p \wedge q$ (sublinhada), deduz-se $q$.
- Linhas 4–5: caso 2, hipótese $q \wedge r$, deduz-se $q$.
- Linha 6: as duas sub-deduções acabaram em $q$, por isso $\vee$E dá $q$ **fora** das sub-deduções (de volta à barra principal).

### Repetição

Numa dedução pode sempre **repetir-se** uma fórmula já obtida (e acessível). É a regra **R**: $\dfrac{\phi}{\phi}\,\mathrm{R}$. Faz falta quando um dos casos de $\vee$E já é a própria conclusão:

**Exemplo resolvido — $(p \wedge q) \vee q \vdash q$ (slide 26)**

| | | |
|---|---|---|
| 1 | $(p \wedge q) \vee q$ | |
| 2 | $\quad p \wedge q$ | |
| 3 | $\quad q$ | $\wedge$E, 2 |
| 4 | $\quad q$ | |
| 5 | $\quad q$ | R, 4 |
| 6 | $q$ | $\vee$E, 1, 2–3, 4–5 |

No segundo caso a hipótese **já é** $q$, mas a sub-dedução tem de "terminar em $q$" com um passo seu; a linha 5 repete a 4.

### Alcance das sub-deduções

**Exemplo resolvido — Uma dedução ERRADA (slide 27)**

| | | |
|---|---|---|
| 1 | $(p \wedge q) \vee (q \wedge r)$ | |
| 2 | $\quad p \wedge q$ | |
| 3 | $\quad p$ | $\wedge$E, 2 |
| 4 | $\quad q$ | $\wedge$E, 2 |
| 5 | $\quad q \wedge r$ | |
| 6 | $\quad q$ | $\wedge$E, 5 |
| 7 | $q$ | $\vee$E, 1, 2–4, 5–6 |
| 8 | $q \wedge p$ | $\wedge$I, 7, 3 |

O passo 8 está **errado**: usa a linha 3 ($p$), que foi deduzida dentro de uma sub-dedução (2–4) que **já terminou**. Esse $p$ só valia "no caso $p \wedge q$"; no outro caso não há $p$ nenhum.
E não há maneira de corrigir: $q \wedge p$ nem sequer é consequência semântica da premissa (contraexemplo $v(p) = F$, $v(q) = V$, $v(r) = V$: $\vDash_v q \wedge r$, logo $\vDash_v (p \wedge q) \vee (q \wedge r)$, mas $\nvDash_v p$, logo $\nvDash_v q \wedge p$). Pela integridade de DN, não existe dedução nenhuma.

**Frequente em exame**

**MUITO IMPORTANTE** (slide 28, em letras grandes no original): uma sub-dedução começa com a introdução de **hipóteses novas**, e tudo o que se deduz lá dentro **depende delas**. Quando a sub-dedução acaba, as hipóteses deixam de estar assumidas e, por isso, **nem as hipóteses nem nada do que se deduziu dentro da sub-dedução pode ser usado** depois.
Regra prática para Fitch: a linha $k$ pode citar a linha $a$ só se $a < k$ **e** todas as barras que estão "abertas" na linha $a$ ainda estão abertas na linha $k$. Uma sub-dedução inteira ($i$–$j$) só pode ser citada a partir da linha logo a seguir ao seu fim, ao nível da barra imediatamente à esquerda.

**Exemplo resolvido — $p \wedge (q \vee r) \vdash (p \wedge q) \vee (p \wedge r)$ (slide 40, a)**

Estratégia: tirar $p$ e $q \vee r$ da premissa; a disjunção pede $\vee$E com conclusão $\gamma = (p \wedge q) \vee (p \wedge r)$.

| | | |
|---|---|---|
| 1 | $p \wedge (q \vee r)$ | |
| 2 | $p$ | $\wedge$E, 1 |
| 3 | $q \vee r$ | $\wedge$E, 1 |
| 4 | $\quad q$ | |
| 5 | $\quad p \wedge q$ | $\wedge$I, 2, 4 |
| 6 | $\quad (p \wedge q) \vee (p \wedge r)$ | $\vee$I, 5 |
| 7 | $\quad r$ | |
| 8 | $\quad p \wedge r$ | $\wedge$I, 2, 7 |
| 9 | $\quad (p \wedge q) \vee (p \wedge r)$ | $\vee$I, 8 |
| 10 | $(p \wedge q) \vee (p \wedge r)$ | $\vee$E, 3, 4–6, 7–9 |

Repara que a linha 5 usa a linha 2 ($p$): é permitido, porque a linha 2 está na barra principal, que continua aberta dentro das sub-deduções. O proibido é o contrário (usar cá fora o que está lá dentro).

**Exemplo resolvido — Exercício 2.4(b) (lab, proplogicproofs.pdf) — encontrar o passo errado**

Dedução dada (sem justificações):

| | |
|---|---|
| 1 | $(p \wedge q) \vee r$ |
| 2 | $\quad p \wedge q$ |
| 3 | $\quad p$ |
| 4 | $\quad p \vee r$ |
| 5 | $\quad r$ |
| 6 | $\quad p \vee r$ |
| 7 | $p \wedge r$ |

Justificação de cada passo: 1 premissa; 2 hipótese; 3 $\wedge$E, 2; 4 $\vee$I, 3; 5 hipótese; 6 $\vee$I, 5. Até aqui está tudo certo.

**O passo 7 está errado.** As duas sub-deduções terminam em $p \vee r$, por isso $\vee$E só permite concluir $p \vee r$, não $p \wedge r$. E nenhuma outra regra dá $p \wedge r$: $p$ (linha 3) e $r$ (linha 5) estão em sub-deduções já fechadas.

**Há uma dedução correta de $p \wedge r$ com a mesma premissa?** Não. Contraexemplo: $v(p) = F$, $v(q) = F$, $v(r) = V$. Então $\models_v r$, logo $\models_v (p \wedge q) \vee r$; e $\not\models_v p$, logo $\not\models_v p \wedge r$. Assim $(p \wedge q) \vee r \not\models p \wedge r$ e, pela integridade de DN, $(p \wedge q) \vee r \not\vdash p \wedge r$.

O que a dedução **mostra** se corrigirmos o passo 7:

| | | |
|---|---|---|
| 1 | $(p \wedge q) \vee r$ | |
| 2 | $\quad p \wedge q$ | |
| 3 | $\quad p$ | $\wedge$E, 2 |
| 4 | $\quad p \vee r$ | $\vee$I, 3 |
| 5 | $\quad r$ | |
| 6 | $\quad p \vee r$ | $\vee$I, 5 |
| 7 | $p \vee r$ | $\vee$E, 1, 2–4, 5–6 |

**Pratica agora — Disjunção (proplogicproofs.pdf)**

- **2.4 (a)** A dedução de $p \land q$ a partir de $p \lor q$ é válida? Se não, qual o passo errado e existe uma dedução correta? Mesma técnica do exemplo 2.4(b) acima.
- **2.11 (h)** $(\delta \lor \phi) \land (\delta \lor \psi) \dashv\vdash \delta \lor (\phi \land \psi)$. Pista: no sentido $\vdash$ há um $\lor$E **dentro** de outro $\lor$E (no caso $\phi$, abre-se $\delta \lor \psi$).

*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

## Regras da negação e do absurdo (F)

**F** representa uma **contradição** (por exemplo $\phi \land \neg\phi$).

**Atenção**

Nota adicional (não estava explícito nos slides): semanticamente, **F** é a fórmula **sempre falsa**: $\not\models_v \mathbf{F}$ para toda a valoração $v$. É o mesmo **F** que aparecia como "cláusula vazia" no algoritmo DLL (Aula 4). Um conjunto $\Sigma$ com $\Sigma \models \mathbf{F}$ é exatamente um conjunto insatisfazível (nenhuma $v$ satisfaz $\Sigma$, porque nenhuma satisfaz **F**).

**Definição — Regras de ¬**

$$\frac{\neg\neg\phi}{\phi}\ \neg\text{E} \qquad\qquad \frac{\begin{array}{c}[\phi]\\ \vdots \\ \mathbf{F}\end{array}}{\neg\phi}\ \neg\text{I}$$

- **¬E** (eliminação da dupla negação): de $\neg\neg\phi$ deduz-se $\phi$. Corresponde a uma das metades do princípio da dupla negação.
- **¬I** (prova **por contradição**): se, **supondo** $\phi$, se deduz uma contradição **F**, então pode deduzir-se $\neg\phi$ a partir das premissas originais (fora da sub-dedução). Cita-se o intervalo da sub-dedução: "¬I, $i$–$j$".

**Definição — Regras de F**

Se **F** não for visto como abreviatura de $\phi \land \neg\phi$, é preciso uma regra para o introduzir (e outra para o usar):

$$\frac{\phi \qquad \neg\phi}{\mathbf{F}}\ \mathbf{F}\text{I} \qquad\qquad \frac{\mathbf{F}}{\phi}\ \mathbf{F}\text{E}$$

- **FI**: se já se deduziu $\phi$ **e** $\neg\phi$, tem-se uma contradição. Cita as duas linhas (por qualquer ordem).
- **FE**: de uma contradição deduz-se **qualquer** fórmula.

Um conjunto de fórmulas $\Sigma$ diz-se **inconsistente** se $\Sigma \vdash \mathbf{F}$.

**Atenção**

Nota adicional: a regra ¬E é a que torna esta lógica **clássica**. Na lógica **intuicionista** (referida na Aula 1) não se aceita $\neg\neg\phi \vdash \phi$ em geral. Por isso, quando uma prova precisa de "chegar a $\phi$ por contradição", o caminho em DN tem **dois** passos: supor $\neg\phi$, chegar a **F**, concluir $\neg\neg\phi$ por ¬I e só depois $\phi$ por ¬E (ver 2.4(c) e o exemplo (d) abaixo).

**Exemplo resolvido — $\phi \vdash \neg\neg\phi$ (slide 31)**

Queremos uma negação ($\neg(\neg\phi)$), por isso supõe-se $\neg\phi$ e procura-se **F**:

| | | |
|---|---|---|
| 1 | $\phi$ | |
| 2 | $\quad\neg\phi$ | |
| 3 | $\quad\mathbf{F}$ | **F** I, 1, 2 |
| 4 | $\neg\neg\phi$ | $\neg$I, 2–3 |

**Exemplo resolvido — Exemplos do slide 40: (b), (c) e (d)**

**(b)** $\neg p \vee \neg q \vdash \neg(p \wedge q)$**.** O objetivo é uma negação: supõe-se $p \wedge q$ e procura-se **F**. Para isso usa-se a premissa (uma disjunção) com $\vee$E, e o $\gamma$ dos dois casos é **F**.

| | | |
|---|---|---|
| 1 | $\neg p \vee \neg q$ | |
| 2 | $\quad p \wedge q$ | |
| 3 | $\quad p$ | $\wedge$E, 2 |
| 4 | $\quad q$ | $\wedge$E, 2 |
| 5 | $\quad\neg p$ | |
| 6 | $\quad\mathbf{F}$ | **F** I, 3, 5 |
| 7 | $\quad\neg q$ | |
| 8 | $\quad\mathbf{F}$ | **F** I, 4, 7 |
| 9 | $\quad\mathbf{F}$ | $\vee$E, 1, 5–6, 7–8 |
| 10 | $\neg(p \wedge q)$ | $\neg$I, 2–9 |

**(c)** $\vdash \neg(p \wedge \neg p)$ (sem premissas). Outra negação: supõe-se $p \wedge \neg p$, que se contradiz a si própria.

| | | |
|---|---|---|
| 1 | $\quad p \wedge \neg p$ | |
| 2 | $\quad p$ | $\wedge$E, 1 |
| 3 | $\quad\neg p$ | $\wedge$E, 1 |
| 4 | $\quad\mathbf{F}$ | **F** I, 2, 3 |
| 5 | $\neg(p \wedge \neg p)$ | $\neg$I, 1–4 |

**Exemplo resolvido — Exemplo do slide 40 (d): $\neg(\neg p \vee q) \vdash p$**

A conclusão $p$ não é uma negação nem tem conectivo para "introduzir". Usa-se **contradição**: supõe-se $\neg p$, chega-se a **F**, tem-se $\neg\neg p$ e por fim $p$. A contradição vem da premissa: de $\neg p$ sai $\neg p \vee q$, o contrário da premissa.

| | | |
|---|---|---|
| 1 | $\neg(\neg p \vee q)$ | |
| 2 | $\quad\neg p$ | |
| 3 | $\quad\neg p \vee q$ | $\vee$I, 2 |
| 4 | $\quad\mathbf{F}$ | **F** I, 3, 1 |
| 5 | $\neg\neg p$ | $\neg$I, 2–4 |
| 6 | $p$ | $\neg$E, 5 |

**Atenção**

Se consultares os apontamentos de Nelma Moreira (`Teoricas/`), há duas gralhas nestas provas: no Exemplo 1.15(c), a linha 6 diz "¬E, 2–4" e deve ler-se "¬E, 5"; no Exemplo 1.21(b), a linha 9 diz "∨I, 3" e deve ler-se "∨I, 8". As versões acima estão verificadas.

**Pratica agora — Negação e F (proplogicproofs.pdf)**

- **2.4 (c)** A dedução de $q$ a partir de $\neg(p \vee \neg q)$ é válida? Se sim, indica a regra de cada passo.
- **2.9 (a)** $\neg\delta,\ \delta \vee \phi \vdash \phi$ é válida? Pista: $\vee$E; no caso $\delta$ há uma contradição, e de **F** tira-se o que se quiser.
- **2.9 (d)** $\delta \vee \psi,\ \neg\psi \vee \phi \vdash \delta \vee \phi$ é válida? Pista: dois $\vee$E, um dentro do outro.
- **2.11 (d)** $\neg(\neg\delta_1 \wedge \neg\delta_2) \dashv\vdash \delta_1 \vee \delta_2$. O sentido $\vdash$ é por contradição (supor $\neg(\delta_1 \vee \delta_2)$), como no exemplo (d).

*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

## Regras da implicação

**Definição — Eliminação de $\rightarrow$ (*modus ponens*)**

$$\frac{\phi \qquad \phi \rightarrow \psi}{\psi}\ {\rightarrow}\text{E}$$

Se já se deduziu $\phi$ e $\phi \rightarrow \psi$, deduz-se $\psi$. É o raciocínio condicional, conhecido por *modus ponens* ("modo que afirma"). Em Fitch citam-se as duas linhas.

**Exemplo resolvido — $p,\ p \rightarrow q,\ p \rightarrow (q \rightarrow r) \vdash r$ (slide 35)**

| | | |
|---|---|---|
| 1 | $p \rightarrow (q \rightarrow r)$ | |
| 2 | $p \rightarrow q$ | |
| 3 | $p$ | |
| 4 | $q \rightarrow r$ | $\rightarrow$E, 1, 3 |
| 5 | $q$ | $\rightarrow$E, 2, 3 |
| 6 | $r$ | $\rightarrow$E, 4, 5 |

**Definição — Introdução de $\rightarrow$ (regra da dedução)**

$$\begin{array}{c}[\phi]\\ \vdots\\ \psi\end{array}\over{\phi \rightarrow \psi}\ {\rightarrow}\text{I}$$

Para deduzir $\phi \rightarrow \psi$: **supõe-se** $\phi$ (abre-se uma sub-dedução) e tenta-se deduzir $\psi$. Se se conseguir, fecha-se a sub-dedução (a suposição é descartada) e conclui-se $\phi \rightarrow \psi$. Cita-se o intervalo: "$\rightarrow$I, $i$–$j$".

**Exemplo resolvido — $(p \vee q) \rightarrow r \vdash p \rightarrow r$ (slide 37)**

| | | |
|---|---|---|
| 1 | $(p \vee q) \rightarrow r$ | |
| 2 | $\quad p$ | |
| 3 | $\quad p \vee q$ | $\vee$I, 2 |
| 4 | $\quad r$ | $\rightarrow$E, 1, 3 |
| 5 | $p \rightarrow r$ | $\rightarrow$I, 2–4 |

### Deduções sem premissas

Com $\rightarrow$I, qualquer dedução com premissas pode ser convertida numa dedução **sem premissas**: em vez de partir da premissa $\phi$, supõe-se $\phi$ numa sub-dedução e no fim aplica-se $\rightarrow$I. Em Fitch, uma dedução sem

premissas começa logo com uma sub-dedução.

**Exemplo resolvido — $\vdash \phi \to \neg\neg\phi$ (slide 39)**

É a dedução $\phi \vdash \neg\neg\phi$ (acima) "embrulhada" numa sub-dedução:

| | | |
|---|---|---|
| 1 | $\phi$ | |
| 2 | $\quad\neg\phi$ | |
| 3 | $\quad\mathbf{F}$ | $\mathbf{F}$ I, 1, 2 |
| 4 | $\neg\neg\phi$ | $\neg$I, 2–3 |
| 5 | $\phi \to \neg\neg\phi$ | $\to$I, 1–4 |

**Definição — Lema da dedução**

Nota adicional (apontamentos de Nelma Moreira, Lema 1.4): em geral,

$$\Sigma \cup \{\phi\} \vdash \psi \quad \text{sse} \quad \Sigma \vdash \phi \to \psi.$$

- ($\Rightarrow$) Dada a dedução de $\psi$ com a premissa extra $\phi$: supõe-se $\phi$ numa sub-dedução, repetem-se lá dentro os passos dessa dedução até $\psi$, e aplica-se $\to$I.
- ($\Leftarrow$) Dada a dedução de $\phi \to \psi$ a partir de $\Sigma$: acrescenta-se $\phi$ às premissas e aplica-se $\to$E.

Por isso $\phi_1, \ldots, \phi_n \vdash \psi$ sse $\vdash \phi_1 \to (\phi_2 \to \cdots (\phi_n \to \psi))$.

**Exemplo resolvido — A tautologia do slide 16, agora formal**

$\vdash (p \vee (q \wedge r)) \to ((p \vee q) \wedge (p \vee r))$. É o raciocínio informal do início da aula, passo a passo: $\to$I por fora, $\vee$E por dentro, e em cada caso $\vee$I + $\wedge$I.

| | | |
|---|---|---|
| 1 | $p \vee (q \wedge r)$ | |
| 2 | $\quad p$ | |
| 3 | $\quad p \vee q$ | $\vee$I, 2 |
| 4 | $\quad p \vee r$ | $\vee$I, 2 |
| 5 | $\quad (p \vee q) \wedge (p \vee r)$ | $\wedge$I, 3, 4 |
| 6 | $\quad q \wedge r$ | |
| 7 | $\quad q$ | $\wedge$E, 6 |
| 8 | $\quad r$ | $\wedge$E, 6 |
| 9 | $\quad p \vee q$ | $\vee$I, 7 |
| 10 | $\quad p \vee r$ | $\vee$I, 8 |
| 11 | $\quad (p \vee q) \wedge (p \vee r)$ | $\wedge$I, 9, 10 |
| 12 | $(p \vee q) \wedge (p \vee r)$ | $\vee$E, 1, 2–5, 6–11 |
| 13 | $(p \vee (q \wedge r)) \to ((p \vee q) \wedge (p \vee r))$ | $\to$I, 1–12 |

**Exemplo resolvido — O raciocínio do Paulo, agora deduzido**

$(p \wedge q) \to r,\ p,\ \neg r \vdash \neg q$. O objetivo é uma negação: supõe-se $q$ e procura-se **F**. Com $p$ e $q$ monta-se $p \wedge q$, que pela primeira premissa dá $r$, contra $\neg r$.

| | | |
|---|---|---|
| 1 | $(p \wedge q) \to r$ | |
| 2 | $p$ | |
| 3 | $\neg r$ | |
| 4 | $\quad q$ | |
| 5 | $\quad p \wedge q$ | $\wedge$I, 2, 4 |
| 6 | $\quad r$ | $\to$E, 1, 5 |
| 7 | $\quad \mathbf{F}$ | **F** I, 6, 3 |
| 8 | $\neg q$ | $\neg$I, 4–7 |

**Exemplo resolvido — Exercício 2.5(a) (lab, proplogicproofs.pdf)**

$(p \wedge q) \to r,\ p \vdash q \to r$. A conclusão é uma implicação: supor $q$, chegar a $r$, $\to$I.

| | | |
|---|---|---|
| 1 | $(p \wedge q) \to r$ | |
| 2 | $p$ | |
| 3 | $\quad q$ | |
| 4 | $\quad p \wedge q$ | $\wedge$I, 2, 3 |
| 5 | $\quad r$ | $\to$E, 1, 4 |
| 6 | $q \to r$ | $\to$I, 3–5 |

**Exemplo resolvido — Exercício 2.5(d) (lab, proplogicproofs.pdf)**

$p \vee \neg q,\ \neg r \to q,\ r \to \neg s,\ s \vdash p$. A conclusão é uma letra só, mas há uma premissa disjunção com $p$ de um lado: $\vee$E com $\gamma = p$.

- **Caso** $p$: é logo $p$ (repetição).
- **Caso** $\neg q$: não há $p$ à vista, por isso mostra-se que este caso é **impossível** (**F**) e usa-se **F**E. A contradição: $\neg q$ e $\neg r \to q$ obrigam a $r$ (se $\neg r$, viria $q$); $r$ dá $\neg s$, contra a premissa $s$.

| | | |
|---|---|---|
| 1 | $p \vee \neg q$ | |
| 2 | $\neg r \to q$ | |
| 3 | $r \to \neg s$ | |
| 4 | $s$ | |
| 5 | $\quad p$ | |
| 6 | $\quad p$ | R, 5 |
| 7 | $\quad \neg q$ | |
| 8 | $\quad\quad \neg r$ | |
| 9 | $\quad\quad q$ | $\to$E, 2, 8 |
| 10 | $\quad\quad \mathbf{F}$ | **F** I, 9, 7 |
| 11 | $\quad \neg\neg r$ | $\neg$I, 8–10 |
| 12 | $\quad r$ | $\neg$E, 11 |
| 13 | $\quad \neg s$ | $\to$E, 3, 12 |
| 14 | $\quad \mathbf{F}$ | **F** I, 4, 13 |
| 15 | $\quad p$ | **F** E, 14 |
| 16 | $p$ | $\vee$E, 1, 5–6, 7–15 |

**Pratica agora — Implicação, parte 1 (proplogicproofs.pdf)**

- **2.4 (d)** e **2.4 (f)** As deduções dadas são válidas? Se não, qual o passo errado e existe uma dedução correta? (Ambas têm um passo errado; numa delas há dedução correta, e tens de a escrever.)
- **2.5 (b)** $q \to p,\ q \to \neg p \vdash \neg q$.
- **2.5 (e)** $p \to (q \vee r),\ q \to s,\ r \to t \vdash p \to (s \vee t)$. Pista: $\to$I por fora, $\vee$E por dentro.
- **2.5 (g)** $(t \to (r \vee p)) \to ((\neg r \vee k) \wedge \neg k) \vdash \neg r$. Pista: supor $r$; para usar a premissa precisas de $t \to (r \vee p)$, que se obtém com $\to$I.
- **Slide 9** Deduz o raciocínio "estudar ou chumbar": $r \to t,\ \neg e \to r,\ \neg t \vdash e \vee r \vee (e \wedge r)$.

As alíneas (c) e (f) da 2.5 são a mesma técnica; não precisas de as fazer.
*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

**Pratica agora — Implicação, parte 2: deduções sem premissas (proplogicproofs.pdf)**

- **2.6 (b)** $\vdash (\delta \to \psi) \to ((\neg\delta \to \neg\phi) \to (\phi \to \psi))$. Três $\to$I encaixados; no fim, para chegar a $\delta$, contradição.
- **2.6 (d)** $\vdash \delta \to (\neg\phi \to \neg(\delta \to \phi))$.
- **2.6 (e)** $\vdash ((\phi \to \delta) \wedge (\delta \to \phi)) \to ((\phi \vee \delta) \to (\phi \wedge \delta))$.
- **2.8 (a)** $\delta \to (\phi \to \psi),\ \neg\psi,\ \delta \vdash \neg\phi$.
- **2.8 (c)** $\vdash (\delta \to (\phi \to \psi)) \to (\phi \to (\delta \to \psi))$.
- **2.8 (e)** $\vdash ((\phi \to \delta) \to \delta) \to ((\delta \to \phi) \to \phi)$. A mais difícil: para usar $(\phi \to \delta) \to \delta$ precisas de $\phi \to \delta$; consegue-se dentro de uma contradição sobre $\neg\phi$.
- **2.8 (g)** $\delta,\ \neg\phi \vdash \neg(\delta \to \phi)$.
- **2.9 (e)** e **2.9 (f)** São válidas? Para a que não for, dá um contraexemplo e usa a integridade.

Ficam de fora a 2.6 (a), (c) e a 2.8 (b), (d), (f): são as mesmas técnicas.
*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

## Regras derivadas e estratégias

A partir das regras base podem obter-se **regras derivadas**: atalhos que correspondem a deduções já feitas, que se podem "colar" no meio de outra. São dos apontamentos de Nelma Moreira (1.4.3.10), não dos slides.

**Atenção**

A folha 2 diz: "In case you use derived rules, you should prove them separately." Ou seja, em exame **podes** usar MT, RA, TE, $\neg\neg$I, mas tens de escrever também a dedução de cada uma (as de baixo). Na dúvida, escreve a prova só com as regras base (as soluções desta folha fazem isso).

### Definição — Regras derivadas e as suas deduções

**Modus tollens** ("modo que nega"):

$$\frac{\phi \to \psi \qquad \neg\psi}{\neg\phi}\ \text{MT}$$

| | | |
|---|---|---|
| 1 | $\phi \to \psi$ | |
| 2 | $\neg\psi$ | |
| 3 | $\quad\phi$ | |
| 4 | $\quad\psi$ | $\to$E, 1, 3 |
| 5 | $\quad\mathbf{F}$ | $\mathbf{F}$ I, 4, 2 |
| 6 | $\neg\phi$ | $\neg$I, 3–5 |

**Introdução da dupla negação** ($\neg\neg$I) e **redução ao absurdo** (RA):

$$\frac{\phi}{\neg\neg\phi}\ \neg\neg\text{I} \qquad\qquad \frac{\begin{array}{c}[\neg\phi]\\ \vdots\\ \mathbf{F}\end{array}}{\phi}\ \text{RA}$$

$\neg\neg$I é o exemplo $\phi \vdash \neg\neg\phi$ da secção da negação. RA abrevia os dois passos "$\neg$I (dá $\neg\neg\phi$) e depois $\neg$E" que já se usaram no exemplo (d) do slide 40.

### Definição — Terceiro excluído (TE): $\vdash \phi \vee \neg\phi$

Sem premissas, só com as regras base. Supõe-se o contrário, $\neg(\phi \vee \neg\phi)$, e mostra-se que dá $\mathbf{F}$: primeiro vê-se que $\phi$ é impossível (daria $\phi \vee \neg\phi$), logo $\neg\phi$; mas $\neg\phi$ também dá $\phi \vee \neg\phi$.

| | | |
|---|---|---|
| 1 | $\quad\neg(\phi \vee \neg\phi)$ | |
| 2 | $\quad\quad\phi$ | |
| 3 | $\quad\quad\phi \vee \neg\phi$ | $\vee$I, 2 |
| 4 | $\quad\quad\mathbf{F}$ | $\mathbf{F}$ I, 3, 1 |
| 5 | $\quad\neg\phi$ | $\neg$I, 2–4 |
| 6 | $\quad\phi \vee \neg\phi$ | $\vee$I, 5 |
| 7 | $\quad\mathbf{F}$ | $\mathbf{F}$ I, 6, 1 |
| 8 | $\neg\neg(\phi \vee \neg\phi)$ | $\neg$I, 1–7 |
| 9 | $\phi \vee \neg\phi$ | $\neg$E, 8 |

### Definição — Contraposição: $\phi \to \psi \dashv\vdash \neg\psi \to \neg\phi$

Sentido $\vdash$ (o outro é igual, com um $\neg$E no fim). É o MT dentro de um $\to$I:

| | | |
|---|---|---|
| 1 | $\phi \to \psi$ | |
| 2 | $\quad\neg\psi$ | |
| 3 | $\quad\quad\phi$ | |
| 4 | $\quad\quad\psi$ | $\to$E, 1, 3 |
| 5 | $\quad\quad\mathbf{F}$ | $\mathbf{F}$ I, 4, 2 |
| 6 | $\quad\neg\phi$ | $\neg$I, 3–5 |
| 7 | $\neg\psi \to \neg\phi$ | $\to$I, 2–6 |

**Definição — Estratégias para encontrar uma dedução (folha 2, página 1)**

Olha para a **conclusão** que falta e para as **premissas/hipóteses** que tens:

1. Conclusão **implicação** $\psi \to \theta$: supõe $\psi$, deduz $\theta$, aplica $\to$I.
2. Conclusão **negação** $\neg\psi$: supõe $\psi$, deduz **F**, aplica $\neg$I.
3. **Por contradição** (quando nada mais serve, ex.: conclusão é uma letra ou uma disjunção): supõe $\neg\phi$ e deduz **F** (depois $\neg$I e $\neg$E). Se há uma negação $\neg\psi$ nas premissas, tenta deduzir $\psi$ para ter **F**.
4. Premissa **disjunção** $\phi \vee \psi$ e queres $\gamma$: supõe $\phi$ e deduz $\gamma$, supõe $\psi$ e deduz $\gamma$, aplica $\vee$E.
5. Hipótese $\neg\phi$ e queres **F**: tenta obter $\phi$ e aplica **F**I.
6. Tens uma implicação $\phi \to \theta$: tenta deduzir $\phi$ e aplica $\to$E para obter $\theta$.

**Cada passo** tem de ser uma premissa, uma hipótese, ou o resultado de uma regra (ou repetição) aplicada a fórmulas já deduzidas e acessíveis. **Não se podem usar equivalências semânticas** (leis de De Morgan, etc.) como passos: se forem precisas, deduzem-se (é isso que pedem as 2.11 e 2.12).

**Exemplo resolvido — Exercício 2.11(e) (lab, proplogicproofs.pdf)**

$\delta_1 \to \delta_2 \dashv\vdash \neg(\delta_1 \wedge \neg\delta_2)$: são duas deduções.
($\vdash$) Conclusão é negação (estratégia 2): supor $\delta_1 \wedge \neg\delta_2$, chegar a **F**.

| | | |
|---|---|---|
| 1 | $\delta_1 \to \delta_2$ | |
| 2 | $\quad \delta_1 \wedge \neg\delta_2$ | |
| 3 | $\quad \delta_1$ | $\wedge$E, 2 |
| 4 | $\quad \neg\delta_2$ | $\wedge$E, 2 |
| 5 | $\quad \delta_2$ | $\to$E, 1, 3 |
| 6 | $\quad$ **F** | **F** I, 5, 4 |
| 7 | $\neg(\delta_1 \wedge \neg\delta_2)$ | $\neg$I, 2–6 |

($\dashv$) Conclusão é implicação (estratégia 1): supor $\delta_1$, chegar a $\delta_2$. $\delta_2$ é uma letra: por contradição (estratégia 3), supor $\neg\delta_2$; com $\delta_1$ monta-se $\delta_1 \wedge \neg\delta_2$, contra a premissa (estratégia 5).

| | | |
|---|---|---|
| 1 | $\neg(\delta_1 \wedge \neg\delta_2)$ | |
| 2 | $\quad \delta_1$ | |
| 3 | $\quad\quad \neg\delta_2$ | |
| 4 | $\quad\quad \delta_1 \wedge \neg\delta_2$ | $\wedge$I, 2, 3 |
| 5 | $\quad\quad$ **F** | **F** I, 4, 1 |
| 6 | $\quad \neg\neg\delta_2$ | $\neg$I, 3–5 |
| 7 | $\quad \delta_2$ | $\neg$E, 6 |
| 8 | $\delta_1 \to \delta_2$ | $\to$I, 2–7 |

**Definição — Formas de raciocínio com nome**

Nota adicional (a 2.1 pede para "identificar a regra de inferência"; os nomes clássicos são estes):

| Nome | Forma | Em DN |
|---|---|---|
| *Modus ponens* | $\phi \to \psi,\ \phi$ logo $\psi$ | $\to$E |
| *Modus tollens* | $\phi \to \psi,\ \neg\psi$ logo $\neg\phi$ | MT |
| Silogismo hipotético | $\phi \to \psi,\ \psi \to \theta$ logo $\phi \to \theta$ | $\to$I com dois $\to$E |
| Silogismo disjuntivo | $\phi \vee \psi,\ \neg\phi$ logo $\psi$ | $\vee$E (2.9 a) |
| Prova por casos | $\phi \to \theta,\ \neg\phi \to \theta$ logo $\theta$ | TE + $\vee$E |
| Contraposição | $\phi \to \psi$ logo $\neg\psi \to \neg\phi$ | acima |

**Falácias** (parecem válidas mas **não são**): *afirmação do consequente* ($\phi \to \psi,\ \psi$ logo $\phi$; contraexemplo $v(\phi) = F$, $v(\psi) = V$) e *negação do antecedente* ($\phi \to \psi,\ \neg\phi$ logo $\neg\psi$; mesmo contraexemplo).

**Pratica agora — Regras derivadas, raciocínios e equivalências (proplogicproofs.pdf)**

- **2.1 (a)** e **(b)** Escreve a forma lógica; se for válido, diz a regra (tabela acima); se não, dá o contraexemplo.
- **2.2** Completa cada argumento com uma conclusão que o torne válido.
- **2.3** Associa cada tautologia (1)–(6) a um dos argumentos (a)–(f).
- **2.7** Formaliza as três hipóteses e deduz "choveu". Pista: estratégia 3 (supor que não choveu).
- **2.11 (g)** $(\phi \vee \psi) \to \delta \dashv\vdash (\phi \to \delta) \wedge (\psi \to \delta)$.

A 2.12 (sem seta na folha) tem mais equivalências do mesmo tipo (De Morgan, distributividade…): só se te sobrar tempo.
*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

## Propriedades de $\vdash$

Nota adicional: os slides não tratam isto explicitamente, mas a 2.10 e a 2.15 (marcadas com seta na folha) pedem para provar ou refutar afirmações gerais sobre $\vdash$. Ferramentas:

- **Para provar que uma afirmação é verdadeira**, constrói-se a dedução pedida **a partir das deduções dadas**: copiam-se as suas linhas (com os números ajustados) e acrescentam-se os passos que faltam. Uma dedução de $\phi$ a partir de $\Sigma$ continua a ser uma dedução de $\phi$ a partir de qualquer $\Sigma' \supseteq \Sigma$: as regras não olham para as premissas que **não** usam.
- **Para provar que é falsa**, instancia-se com fórmulas concretas e, para mostrar o "$\nvdash$", usa-se um contraexemplo + **integridade** (secção "Semântica versus dedução").

**Exemplo resolvido — Exercício 2.10(a) e (b) (lab, proplogicproofs.pdf)**

**(a) Se $\vdash \psi$, então $\delta_1, \ldots, \delta_n \vdash \psi$ — VERDADEIRO.**
Seja $\phi_1, \ldots, \phi_m$ (com $\phi_m = \psi$) uma dedução de $\psi$ sem premissas. Escrevem-se $\delta_1, \ldots, \delta_n$ como premissas (linhas $1, \ldots, n$) e, a seguir, as linhas $\phi_1, \ldots, \phi_m$, somando $n$ a todos os números de linha das justificações. Cada passo $\phi_i$ continua justificado pela mesma regra, aplicada às mesmas fórmulas (que continuam acessíveis: as premissas novas estão na barra principal e não fecham nenhuma sub-dedução). A última linha é $\psi$. Logo $\delta_1, \ldots, \delta_n \vdash \psi$. ■
**(b) Se $\delta_1, \ldots, \delta_n \vdash \psi$, então $\vdash \psi$ — FALSO.**
Contraexemplo: $n = 1$, $\delta_1 = p$, $\psi = p$. Tem-se $p \vdash p$:

| | | |
|---|---|---|
| 1 | $p$ | |
| 2 | $p$ | R, 1 |

Mas $\nvdash p$: com $v(p) = F$ tem-se $\not\vDash_v p$, logo $\not\vDash p$ ($p$ não é tautologia); se houvesse uma dedução $\vdash p$, pela integridade de DN ter-se-ia $\vDash p$. Logo $\nvdash p$. ■

**Pratica agora — Propriedades de $\vdash$ (proplogicproofs.pdf)**

- **2.10 (e)** Se $\delta_1, \delta_2 \vdash \phi \to \psi$ e $\delta_1 \vdash \phi$, então $\delta_1, \delta_2 \vdash \psi$.
- **2.10 (f)** Se $\delta_1, \delta_2 \vdash \phi \to \psi$ e $\delta_1 \vdash \phi$, então $\delta_1 \vdash \psi$. (Compara com a (e): só muda o lado direito.)
- **2.10 (g)** Se $\delta_1 \vdash \phi \to \psi$ e $\delta_2 \vdash \phi$, então $\delta_1, \delta_2 \vdash \psi$.
- **2.15 (a)** e **(b)** Se para **qualquer** $\delta$ se tem $\Sigma \vdash \delta$ ou $\Gamma \vdash \delta$, então $\Sigma$ ou $\Gamma$ é insatisfazível, e $\Sigma \cup \Gamma$ é insatisfazível. Pista: escolhe $\delta = \mathbf{F}$.

As restantes alíneas da 2.10 são a mesma técnica. As 2.13, 2.14 e 2.16 (sem seta) também.
*Soluções, uma alínea de cada vez:* `SOLUCOES.html`

Os exercícios 2.17–2.22 da folha (*tableaux* semânticos e resolução) ainda não foram dados nas teóricas e ficam fora do resumo por agora.